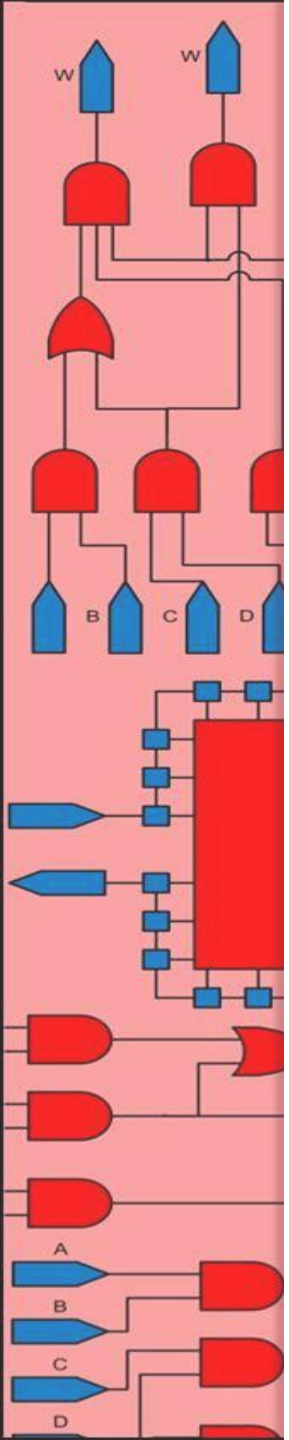
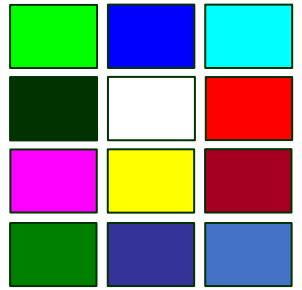
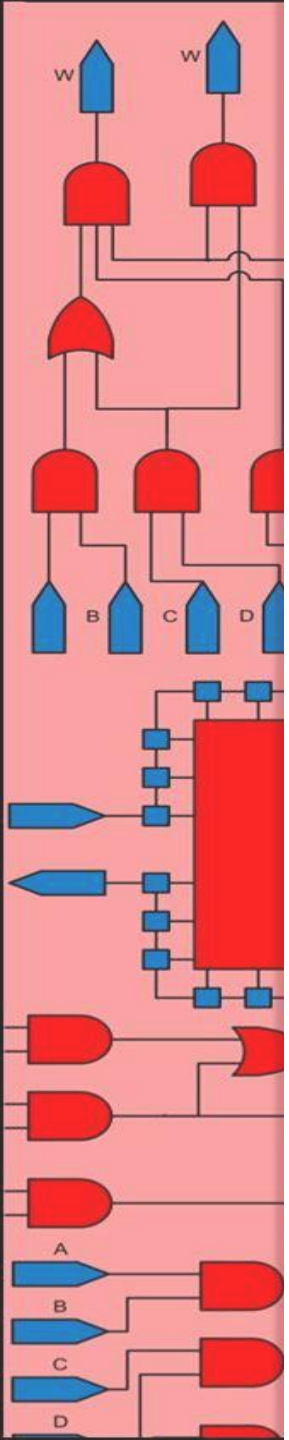




Topic 5

RTL Methodology



Outline

5- RTL Methodology

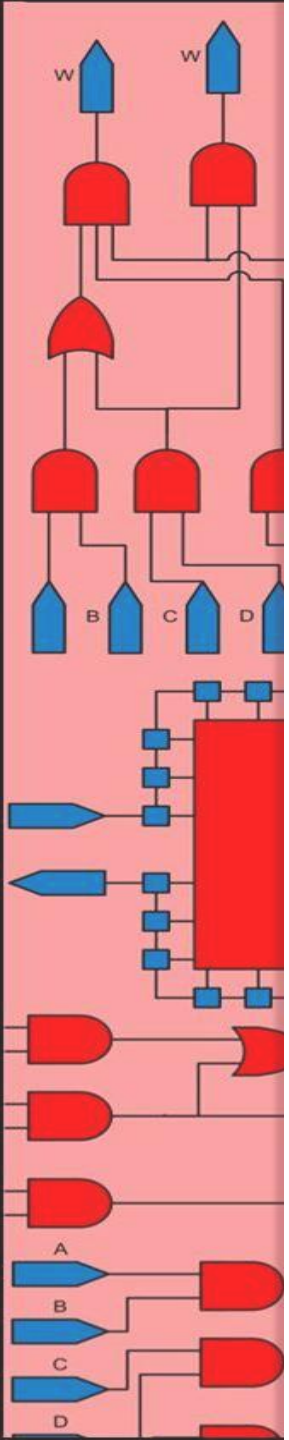
Basic Design Flow

Serial Adder

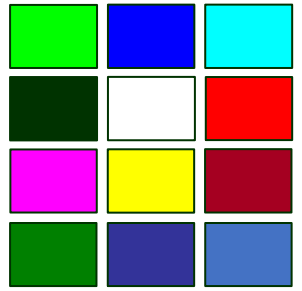
Divider Circuit

Sequential Multiplier

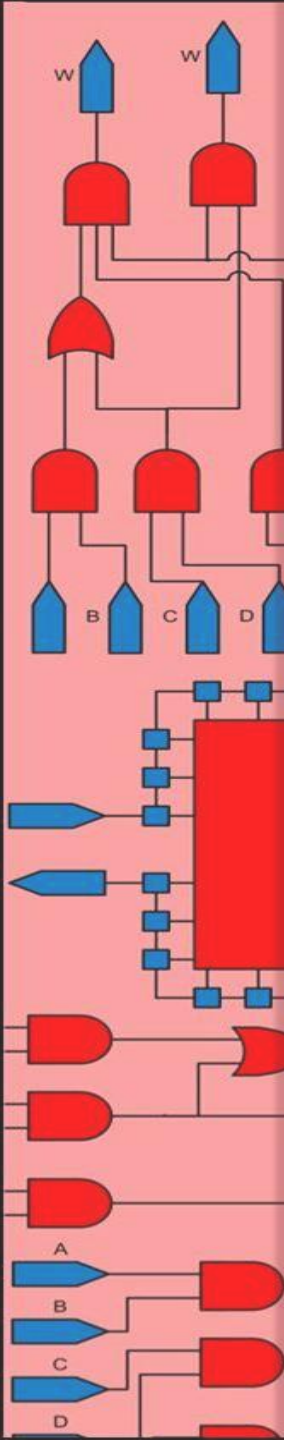
Exponential Accelerator

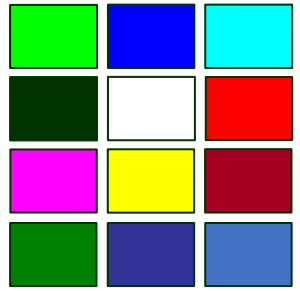


Basic Design Flow



- **Step 1:** Understanding the problem
- **Step 2:** Find a hardware-oriented algorithm for solving the problem
- **Step 3:** Datapath - Controller partitioning
  - Place all the required combinational and sequential components in the datapath
  - Decide on the bussing structure interconnecting sequential and combinational components
  - Decide what control signals components and buses of a datapath require
  - Considering RTL timing design control FSM(s) responsible for issuing datapath and external control signals
- **Step 4:** Implement the design in SystemSystemVerilog





Outline

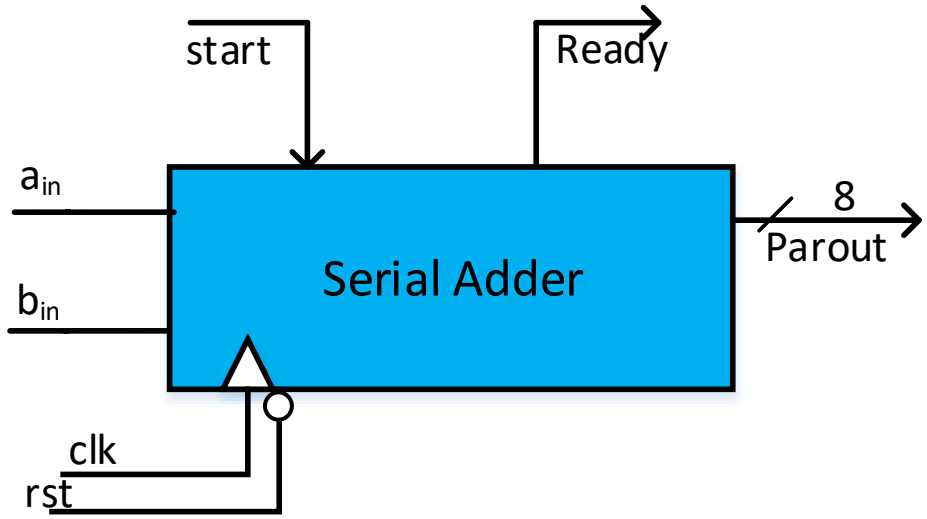
5- RTL Methodology

- Basic Design Flow
- **Serial Adder**
- Divider Circuit
- Sequential Multiplier

Finite State Machine

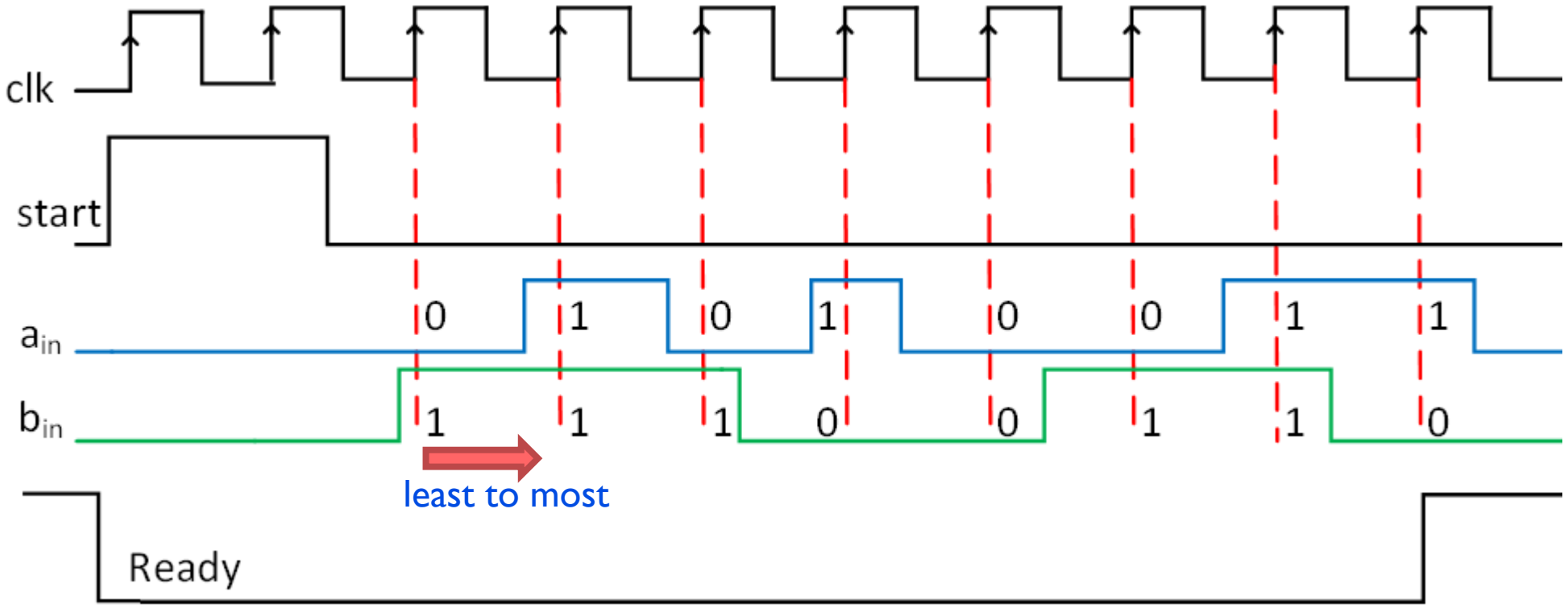
Serial Adder

- After a complete pulse on start, data on a_{in} and b_{in} are serially added
- After 8 clock cycles, the sum result, *Parout*, is ready
- Ready signal is issued and will be asserted until the next start



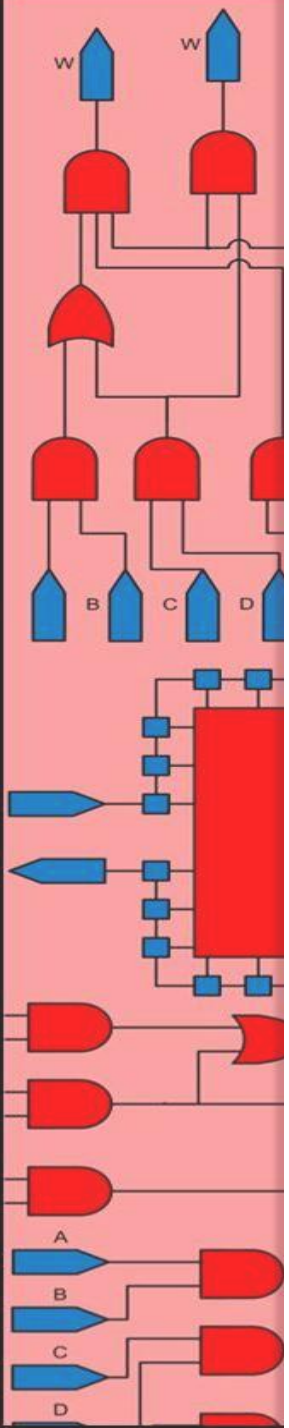
Finite State Machine

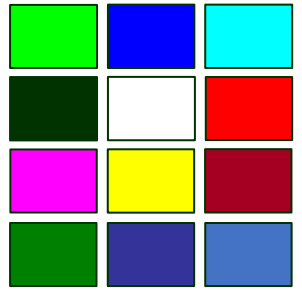
Serial Adder

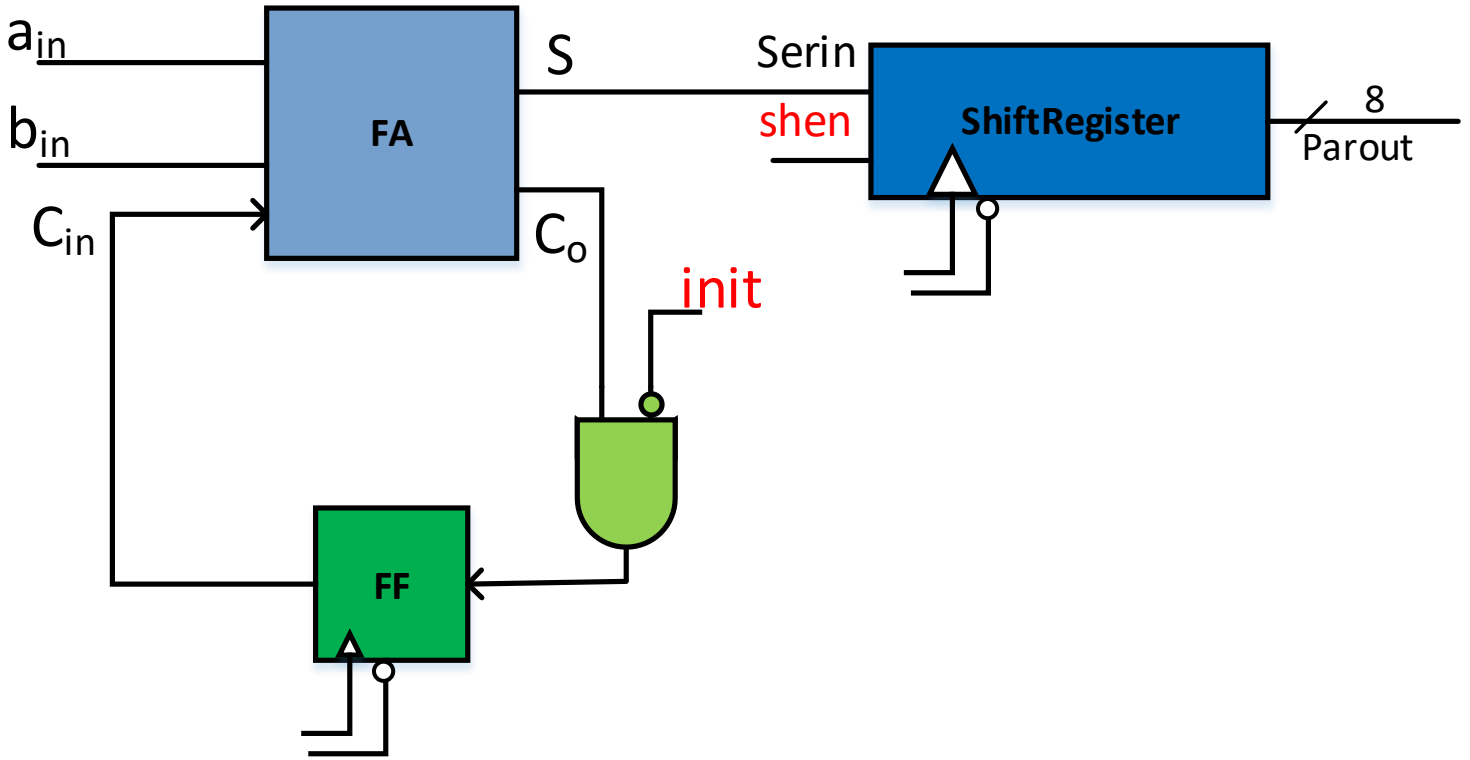


Finite State Machine

Serial Adder - Datapath

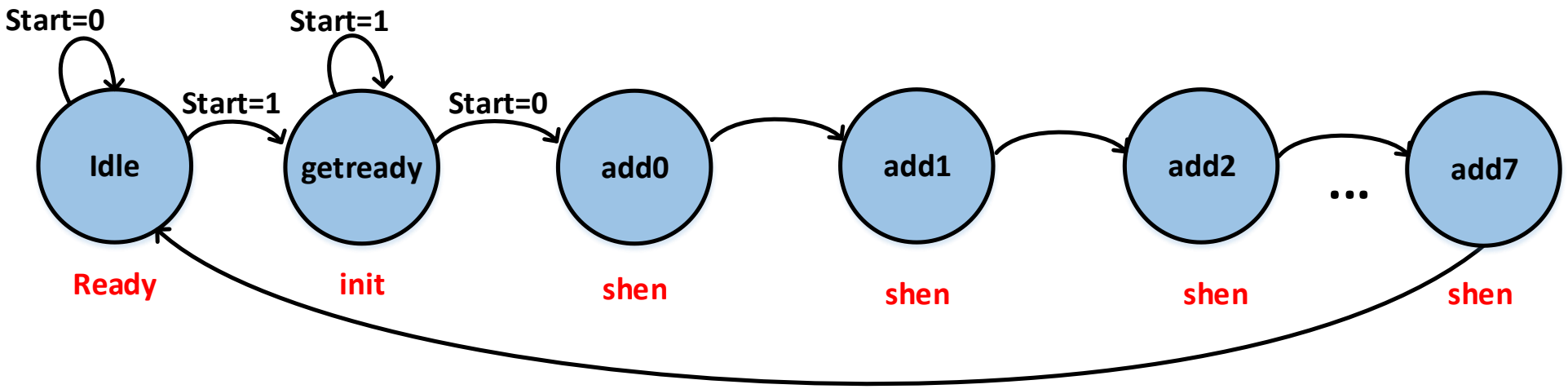


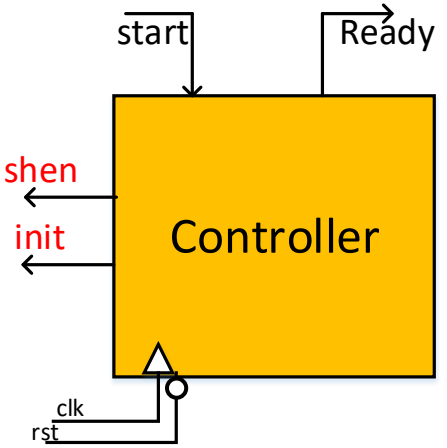




Finite State Machine

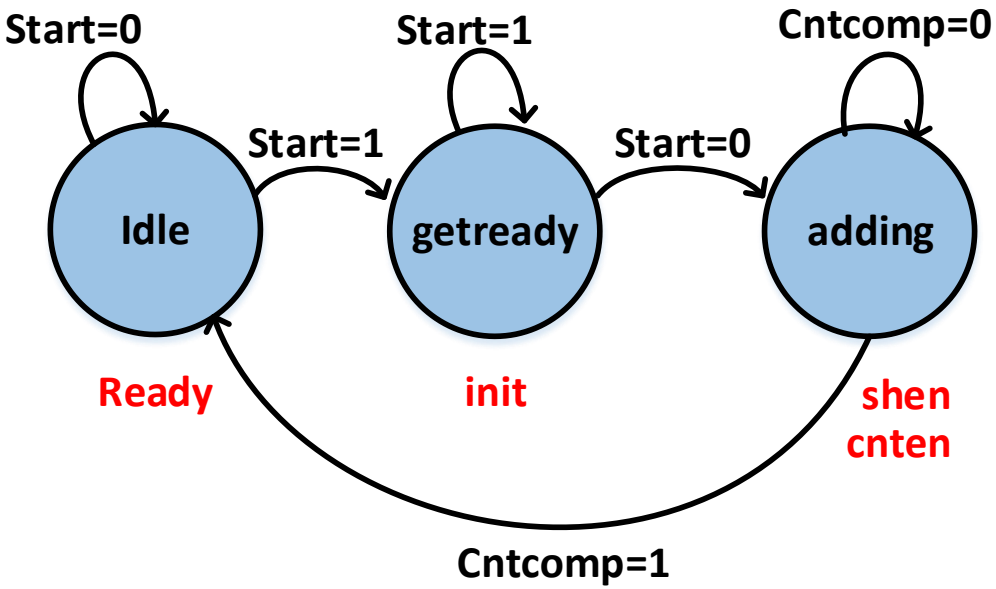
Serial Adder - Controller

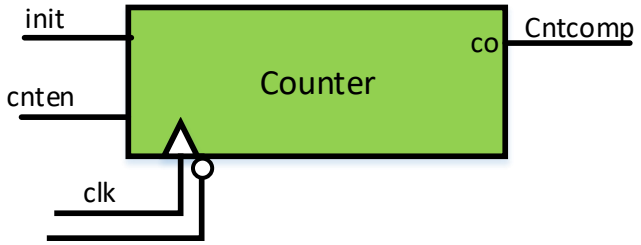


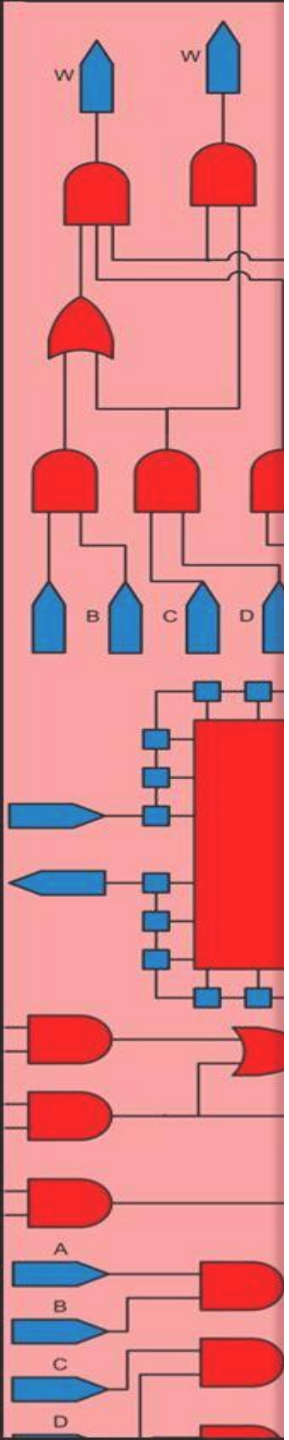


Finite State Machine

Serial Adder - Controller







Outline

5- RTL Methodology

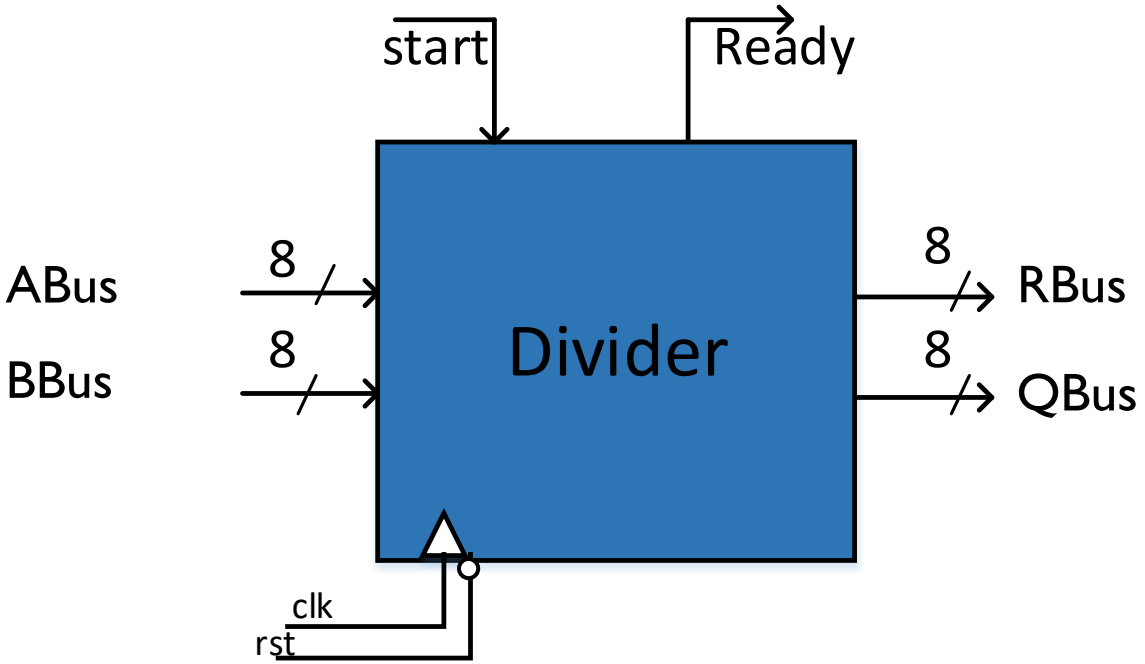
- Basic Design Flow
- Serial Adder
- **Divider Circuit**
- Sequential Multiplier
- Exponential Accelerator

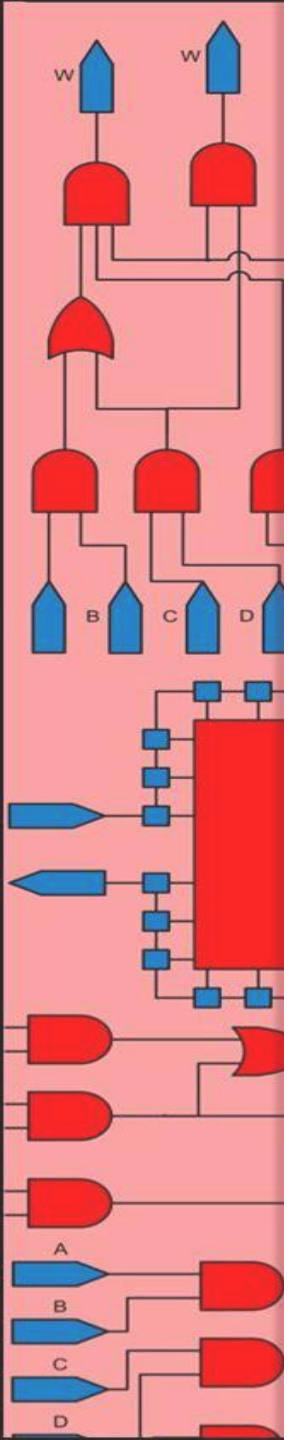
Divider Circuit

- Let's exercise this design flow with a simple divider circuit

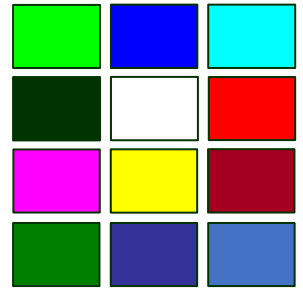
Step 1.

Find what the problem wants you to solve



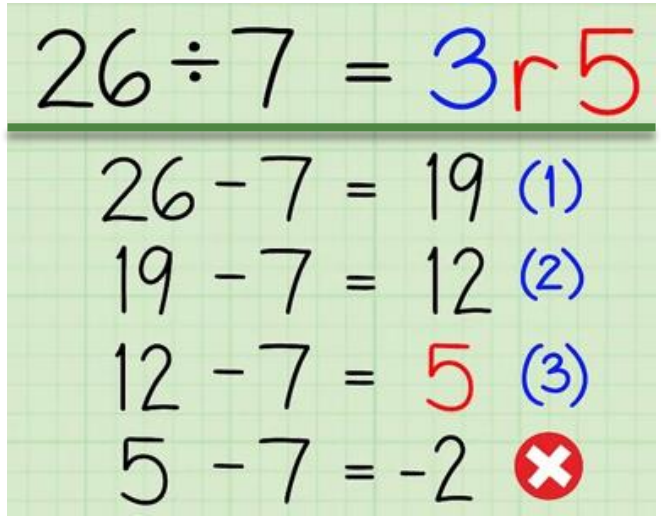


Divider Circuit



Step 2. Find an algorithm for the solution

- **Step 2.1:** Subtract the divisor from the dividend.
- **Step 2.2:** Keep subtracting until you cannot subtract anymore
- **Step 2.3:** keep a count of how many subtractions you can do

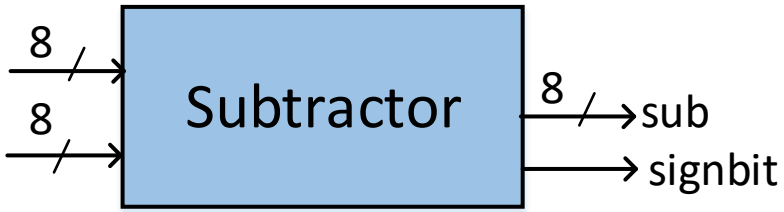


Step 2.2 gives you the remainder
Step 2.3 gives you the quotient

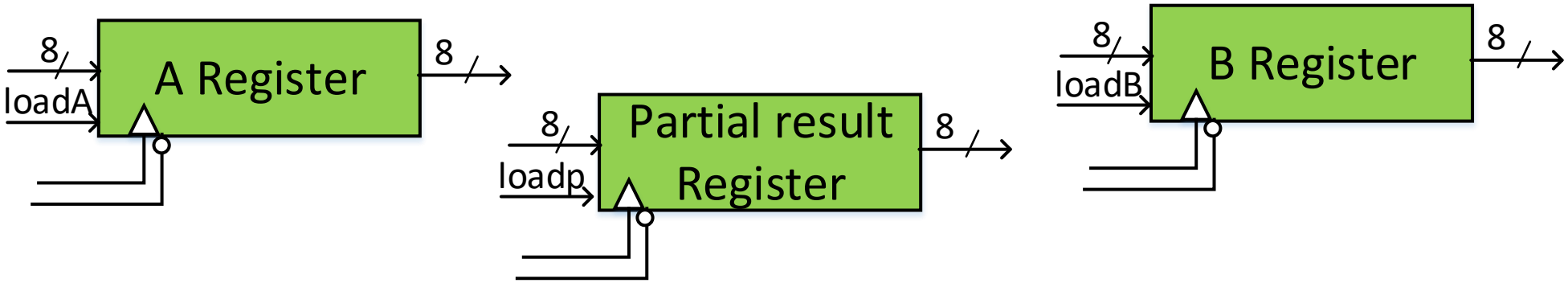
Divider Circuit

Step 3. Datapath - Controller Partitioning

- Subtract the divisor from the dividend (WHAT: Datapath)



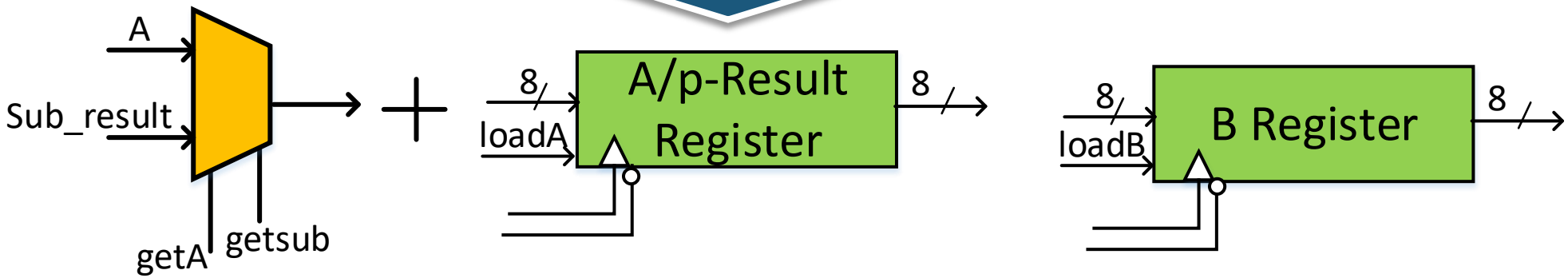
- Keep subtracting the divisor (WHAT: Datapath)



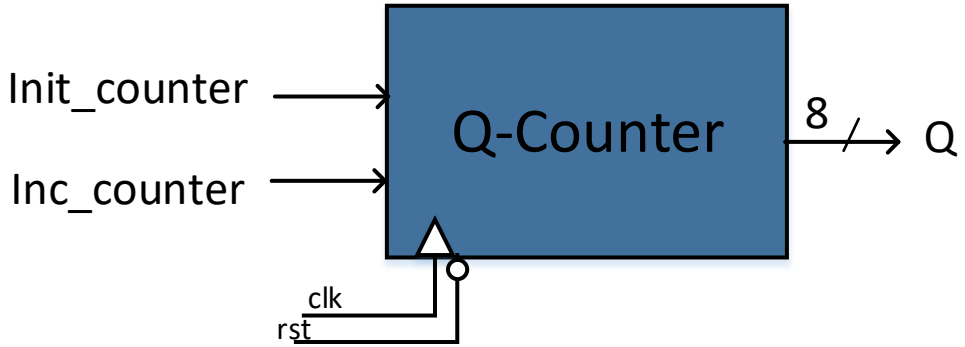
Divider Circuit

Step 3. Datapath - Controller Partitioning

A Better Choice

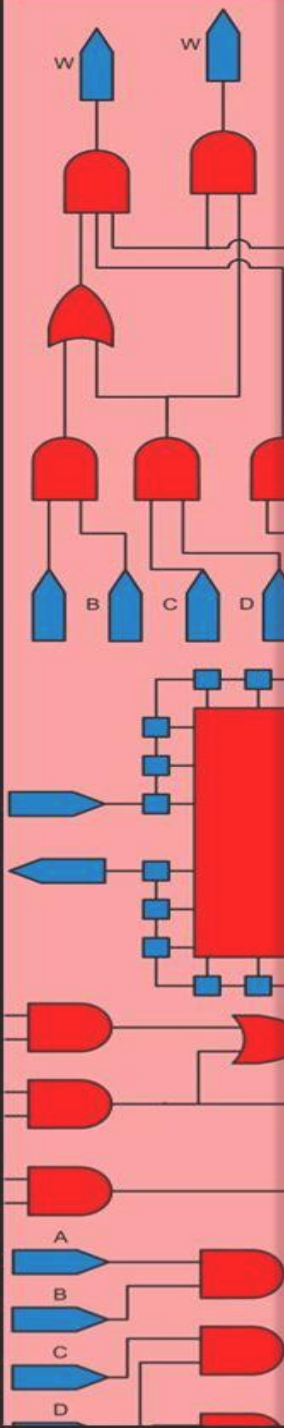


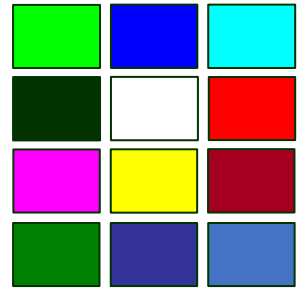
- Count how many times you subtracted the divisor (WHAT : Datapath)



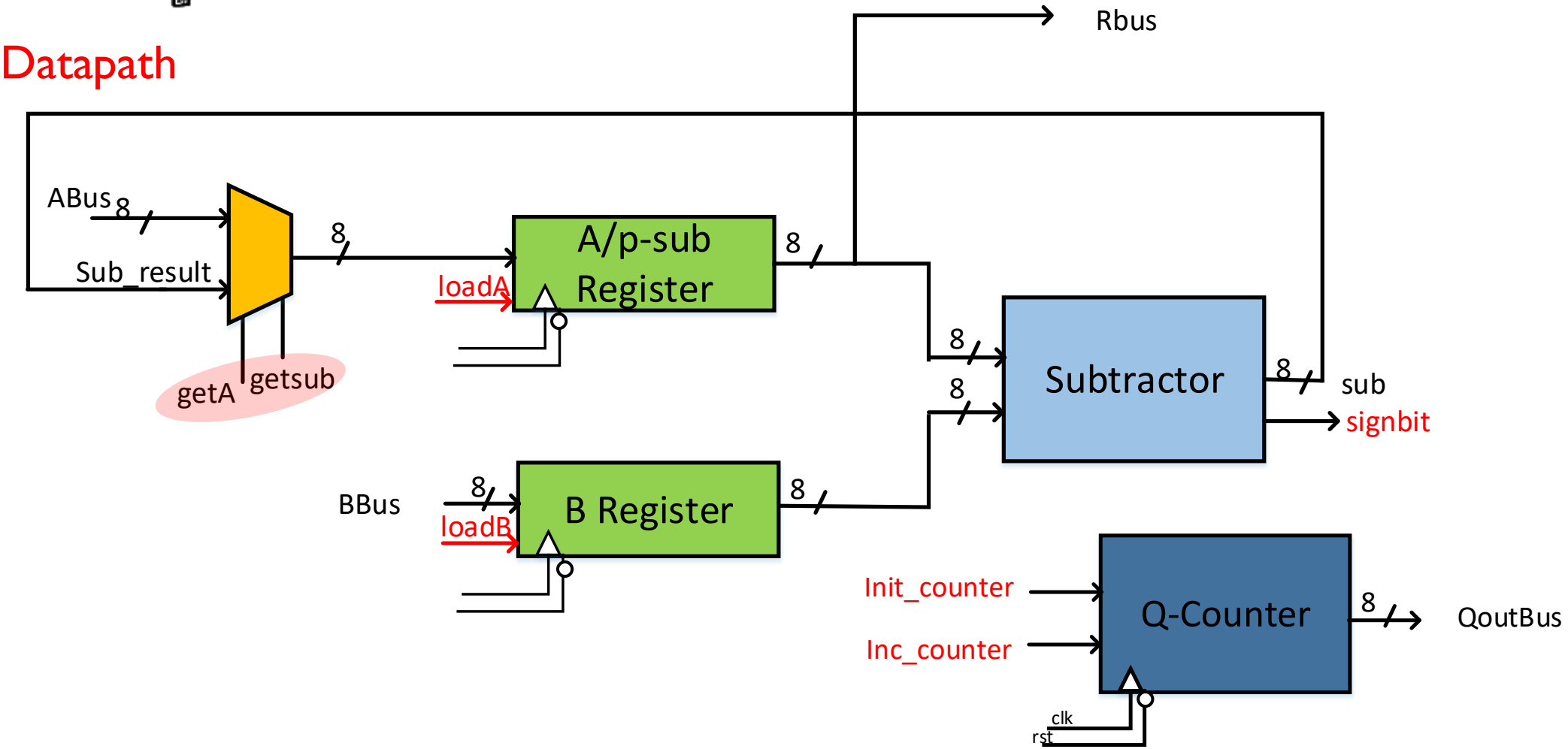
Divider Circuit

Step 3. Datapath- Controller Partitioning



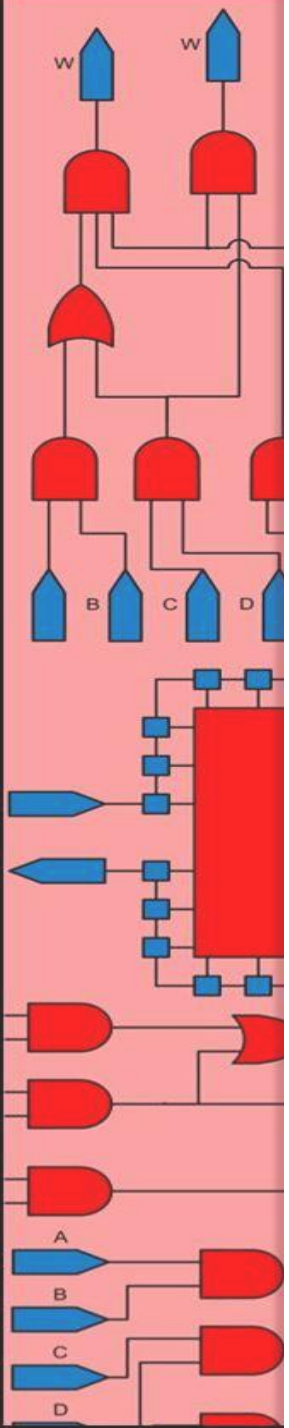


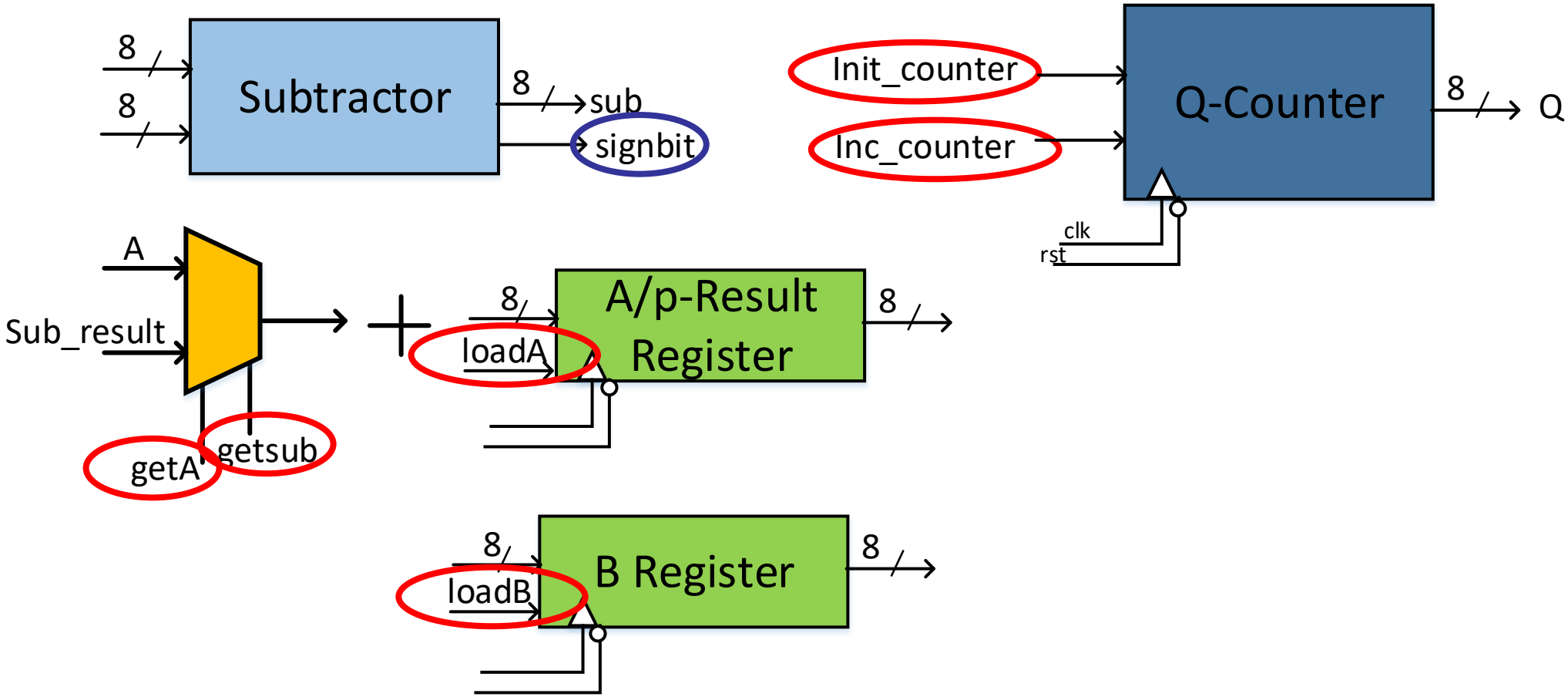
Datapath



Divider Circuit

Step 3. Datapath- Controller Partitioning

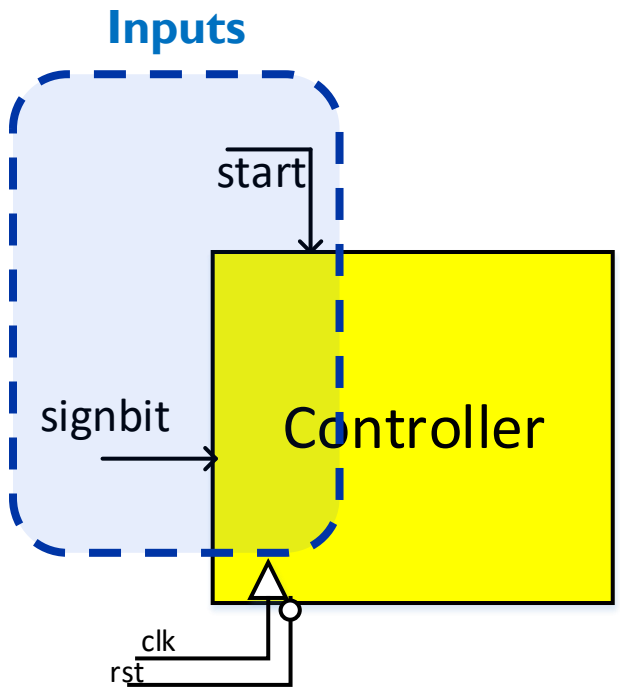




Divider Circuit

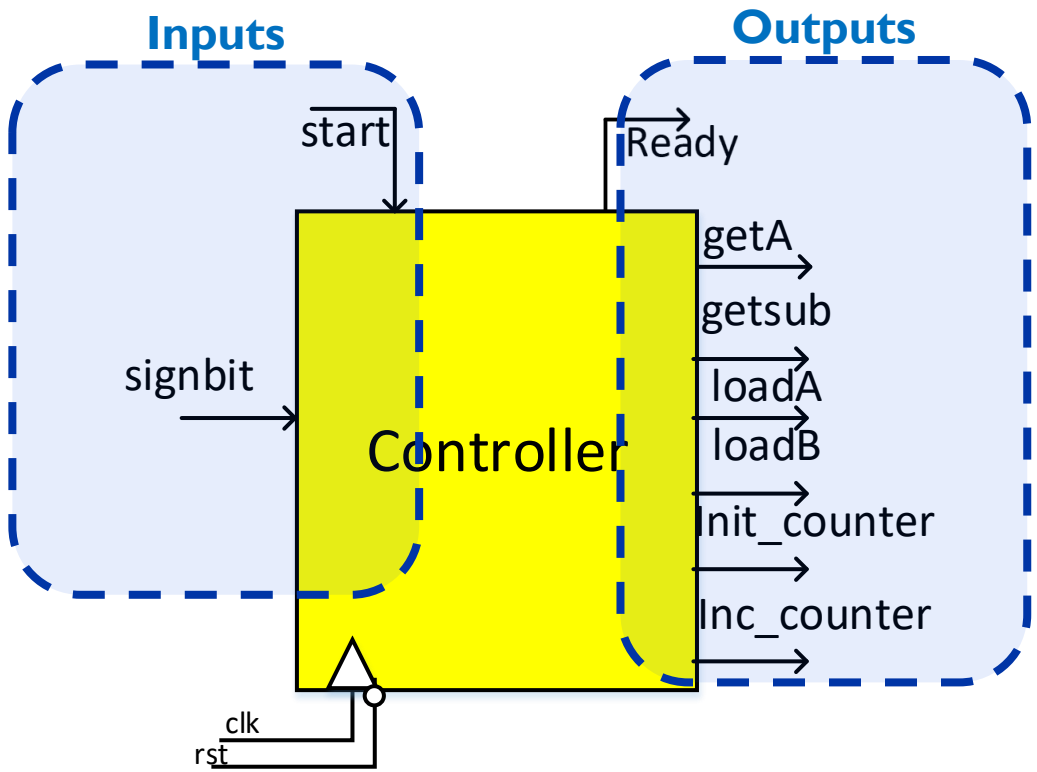
Step 3. Datapath - Controller Partitioning

- Repeat until you cannot subtract anymore (WHEN : Controller)
  - Stop when subtraction result's sign-bit is one.



Divider Circuit

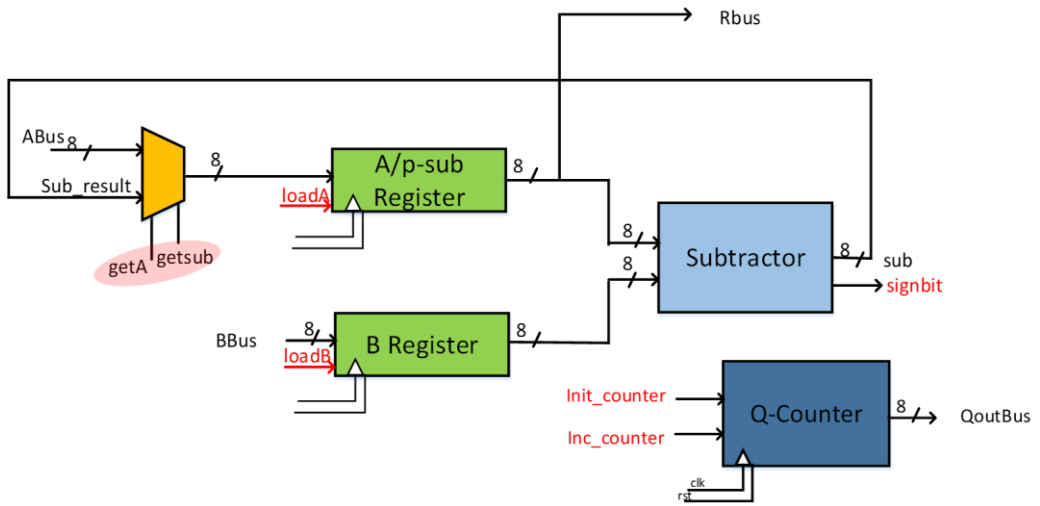
Step 3. Datapath- Controller Partitioning



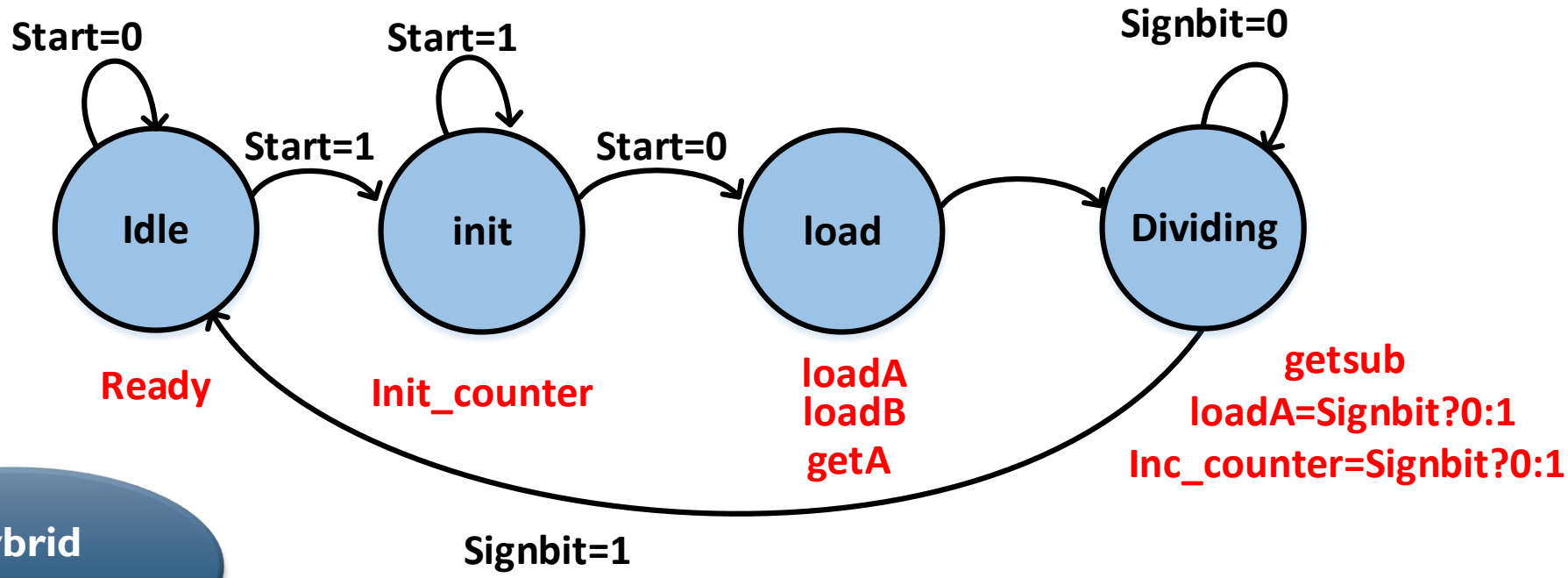
Divider Circuit

Step 3.

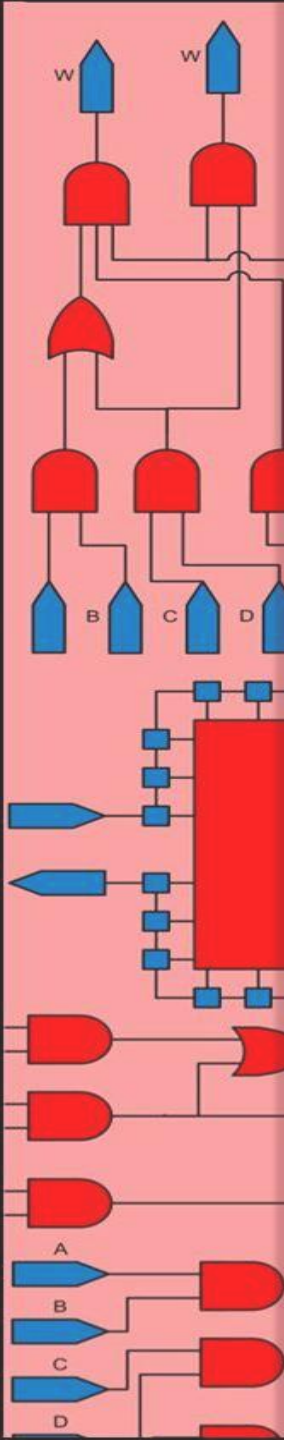
Datapath- Controller Partitioning



Controller



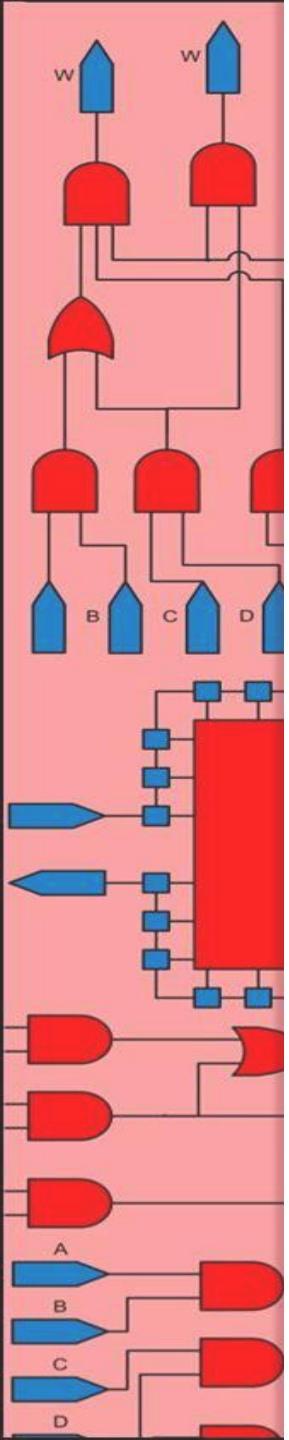
Hybrid Controller



Outline

5- RTL Methodology

- Basic Design Flow
- Serial Adder
- Divider Circuit
- **Sequential Multiplier**
- Exponential Accelerator



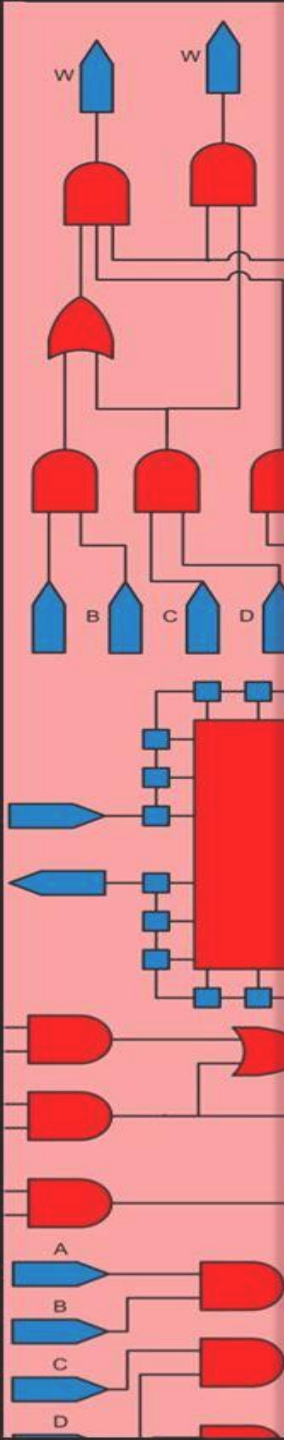
Sequential Multiplier

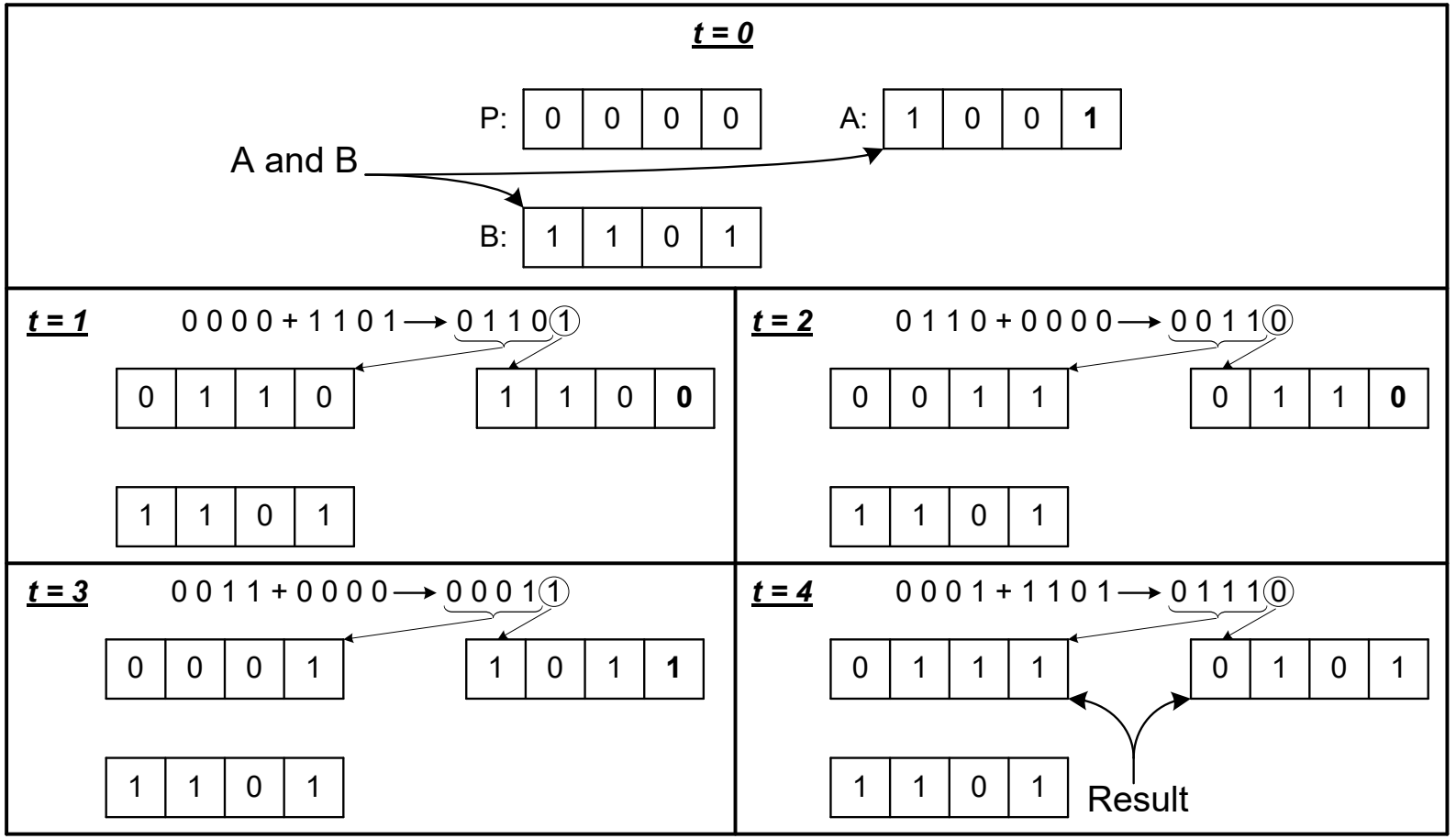
Shift-Add Multiplier

```
B:  1 0 1 1 0 1 1 0
A:  1 0 0 1 0 1 0 0
    ---------------
    0 0 0 0 0 0 0 0
  0 0 0 0 0 0 0 0
1 0 1 1 0 1 1 0
0 0 0 0 0 0 0 0
  1 0 1 1 0 1 1 0
    0 0 0 0 0 0 0 0
      0 0 0 0 0 0 0 0
        1 0 1 1 0 1 1 0
    ---------------------
1 1 0 1 0 0 1 0 0 1 1 1 0 0 0
```

Sequential Multiplier

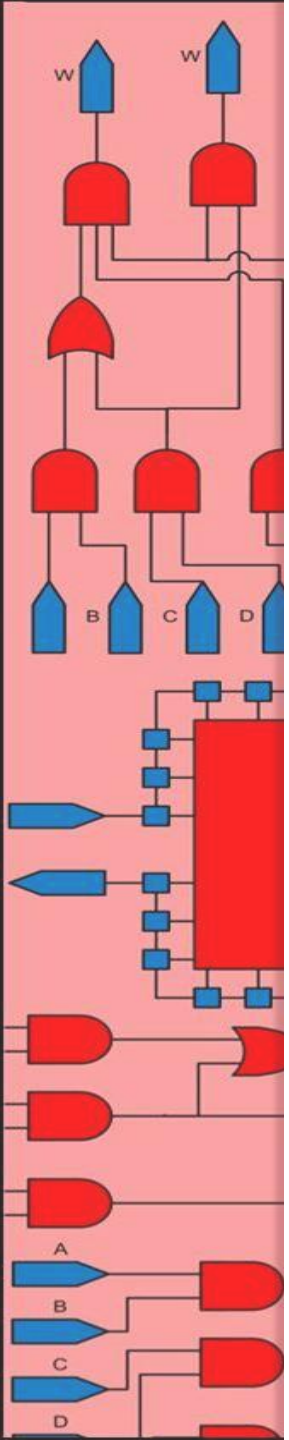
Shift-Add Multiplier

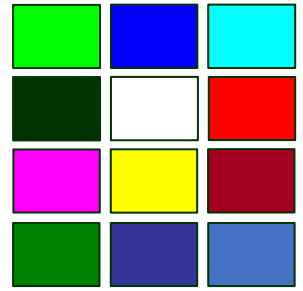


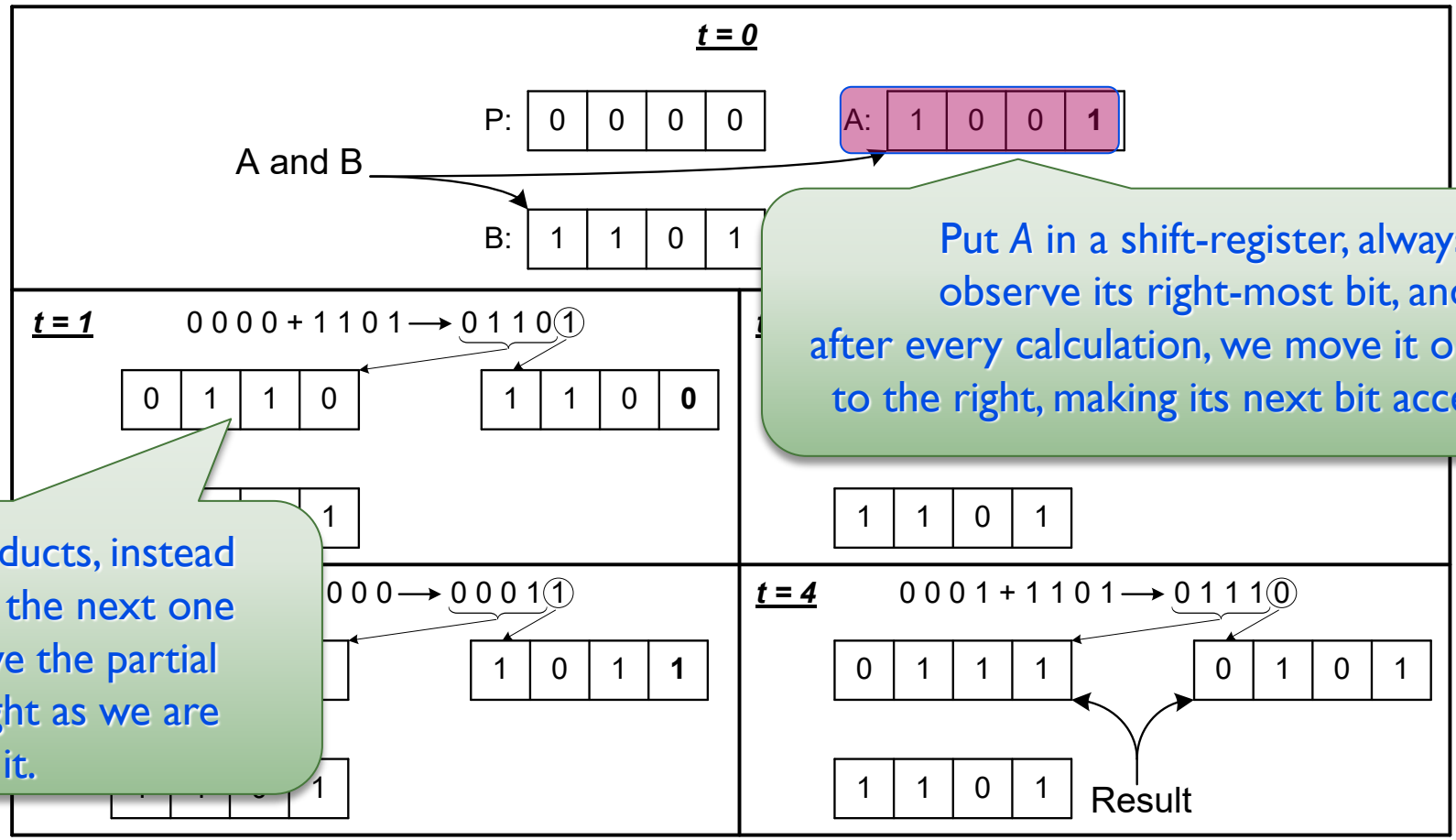


Sequential Multiplier

Shift-Add Multiplier

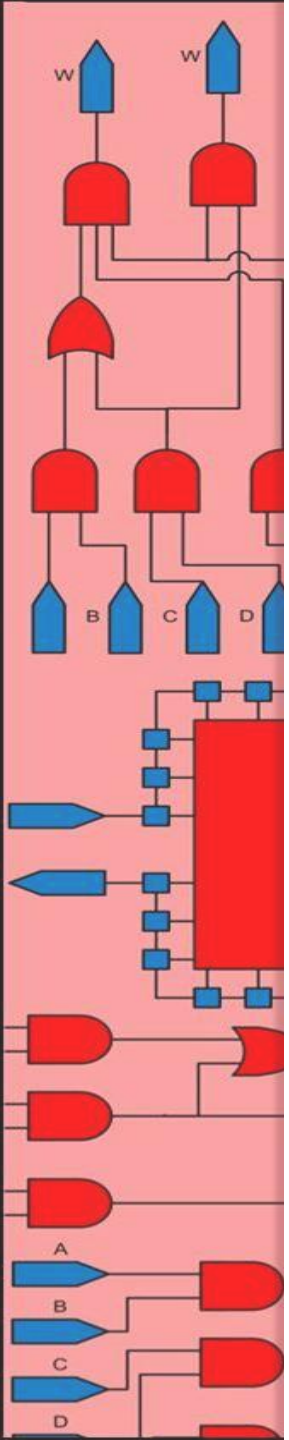






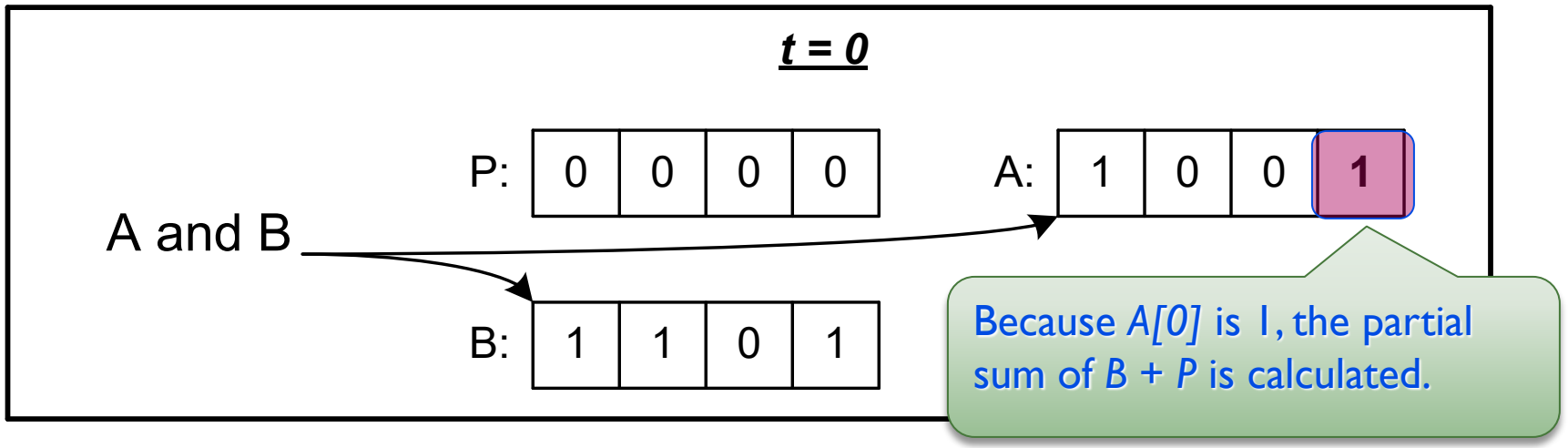
Put *A* in a shift-register, always observe its right-most bit, and after every calculation, we move it one place to the right, making its next bit accessible.

For the partial products, instead of writing one and the next one to its left, we move the partial product to the right as we are writing it.



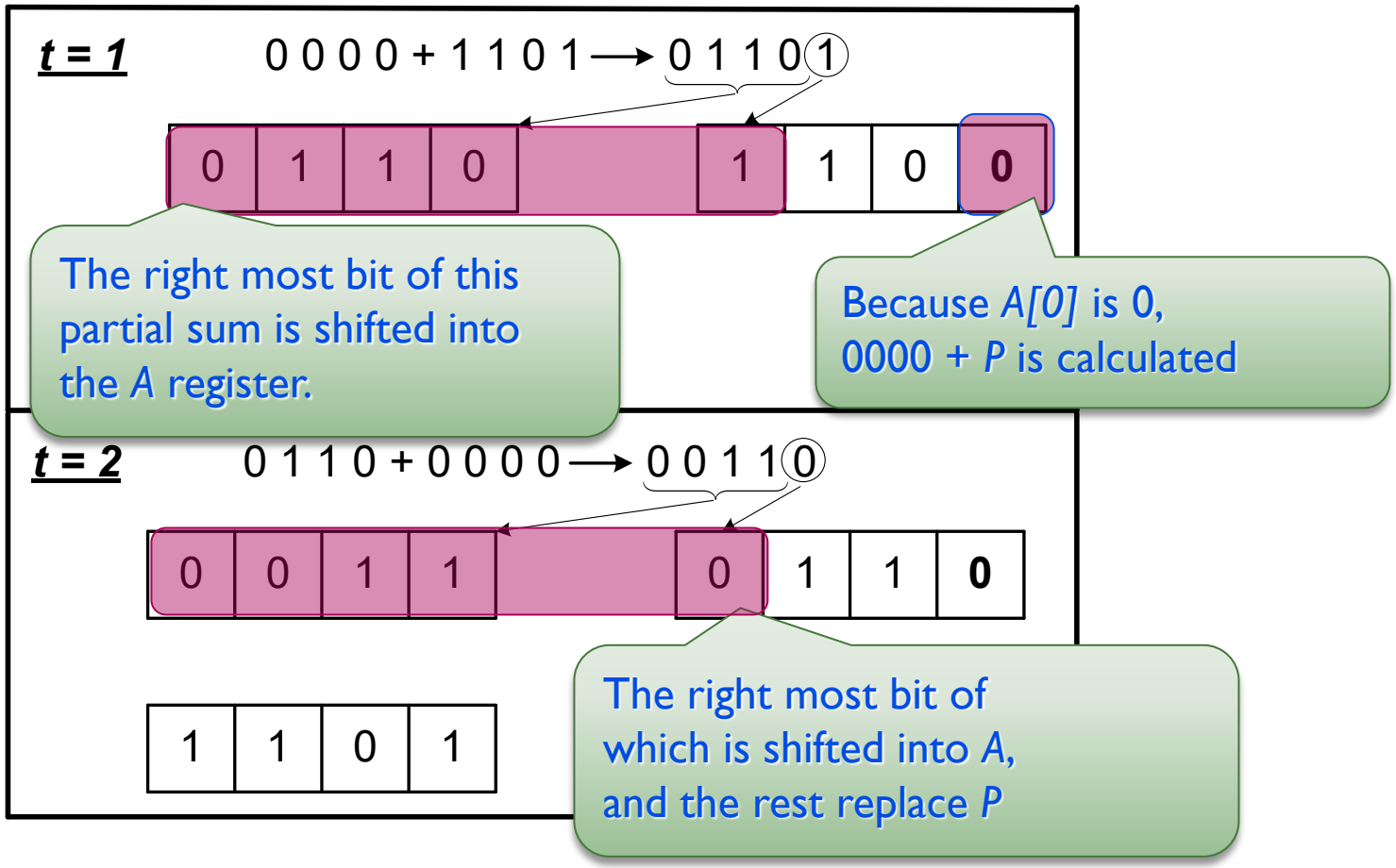
Sequential Multiplier

Shift-Add Multiplier



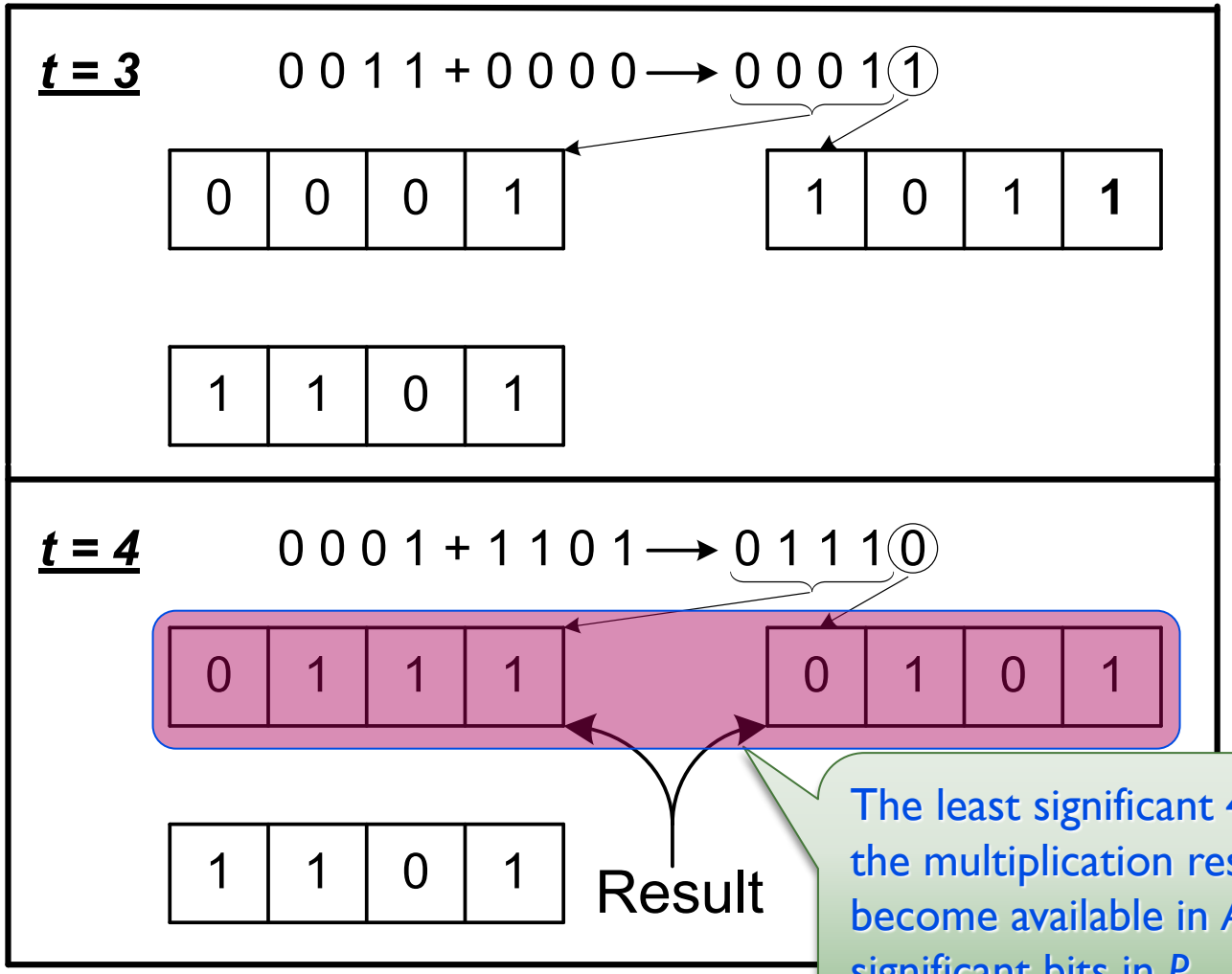
Sequential Multiplier

Shift-Add Multiplier



Sequential Multiplier

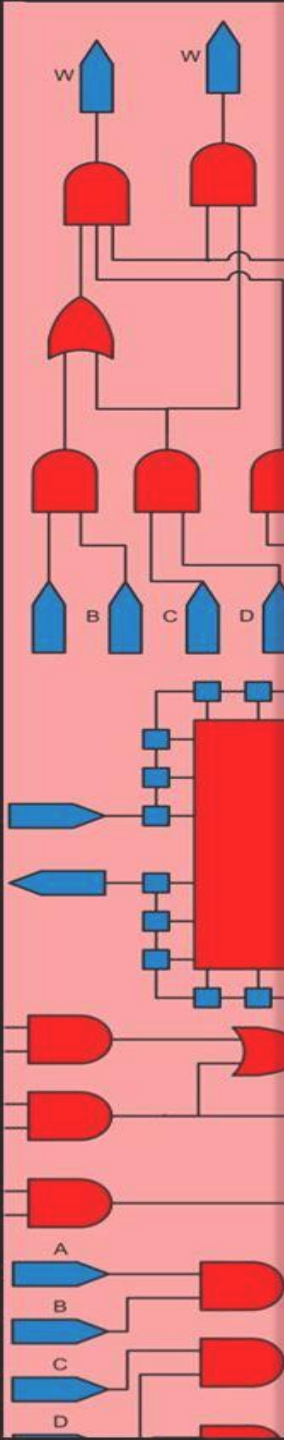
Shift-Add Multiplier

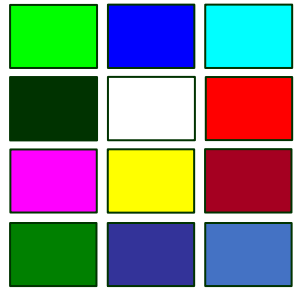


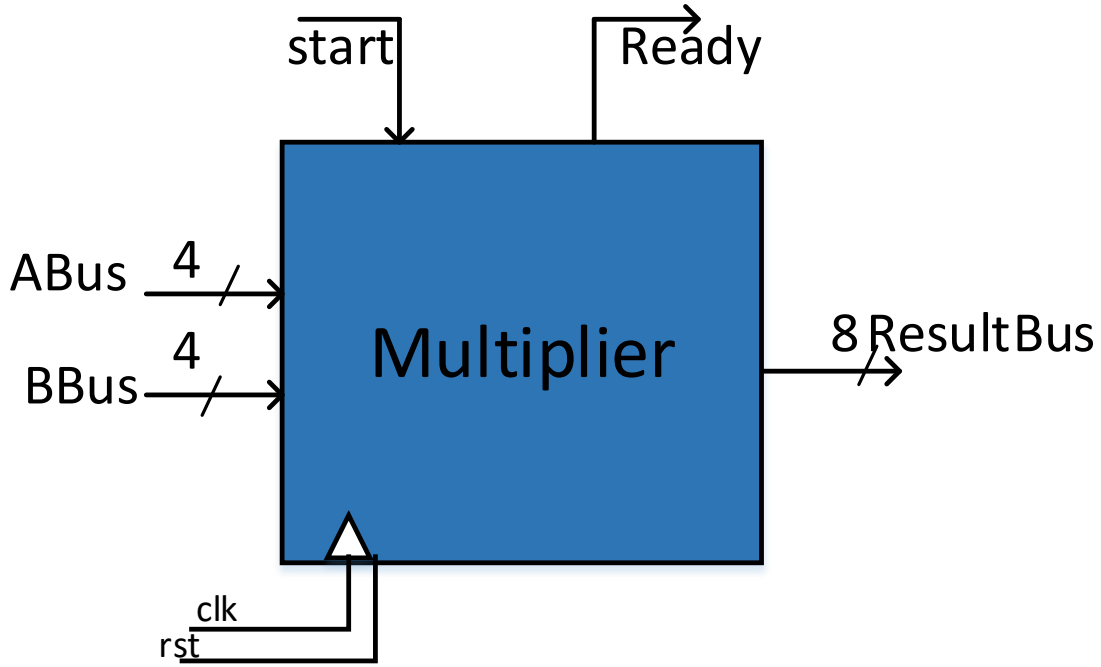
The least significant 4 bits of the multiplication result become available in *A* and the most-significant bits in *P*.

Sequential Multiplier

Shift-Add Multiplier

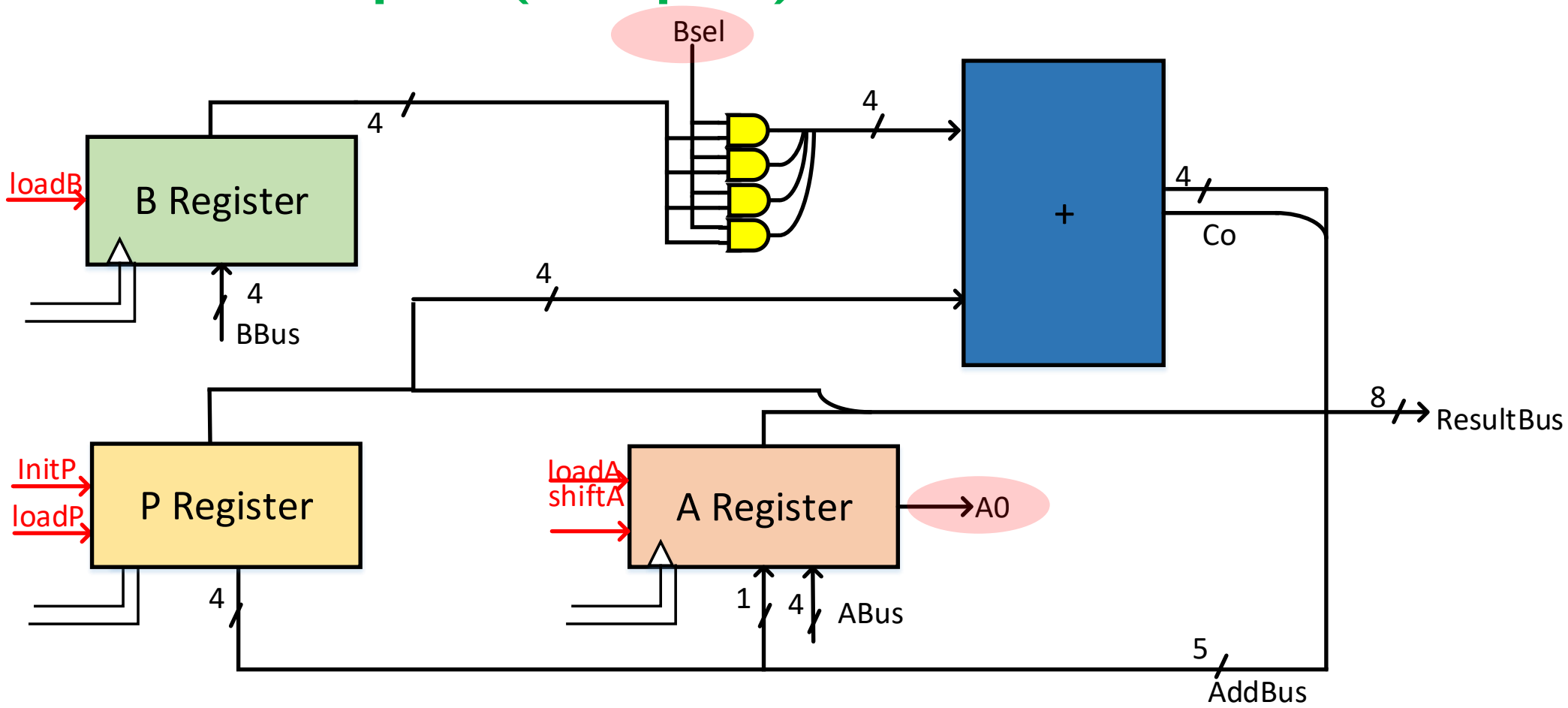






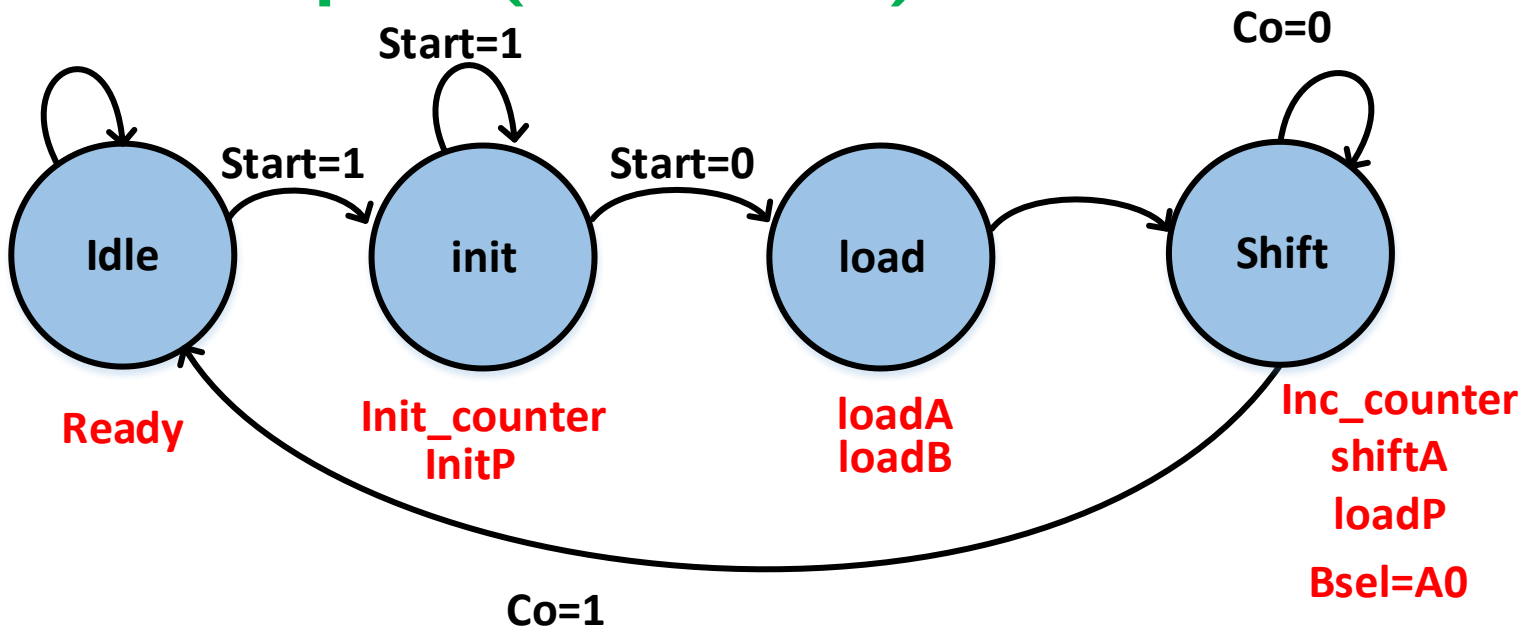
Sequential Multiplier

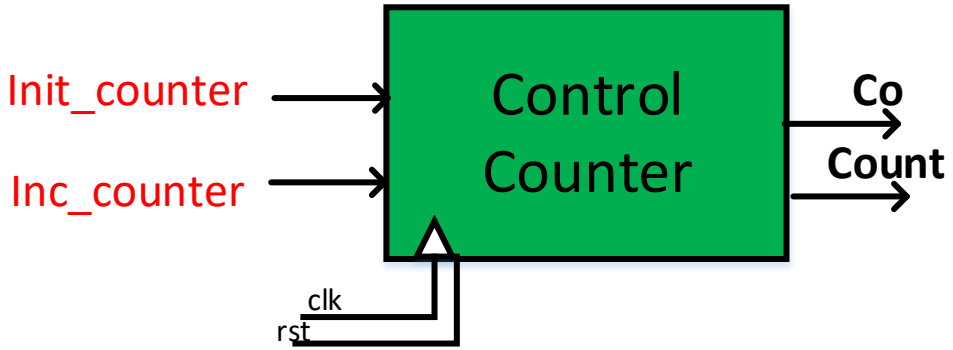
Shift-Add Multiplier (Datapath)

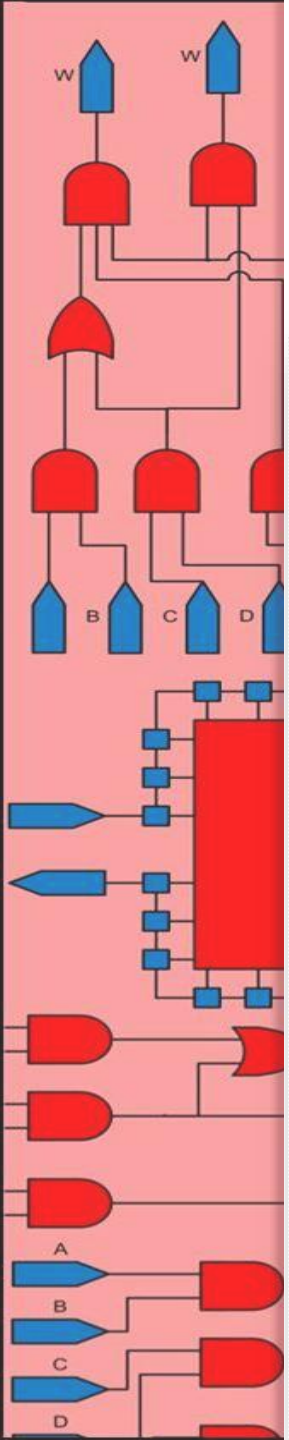


Sequential Multiplier

Shift-Add Multiplier (Controller)

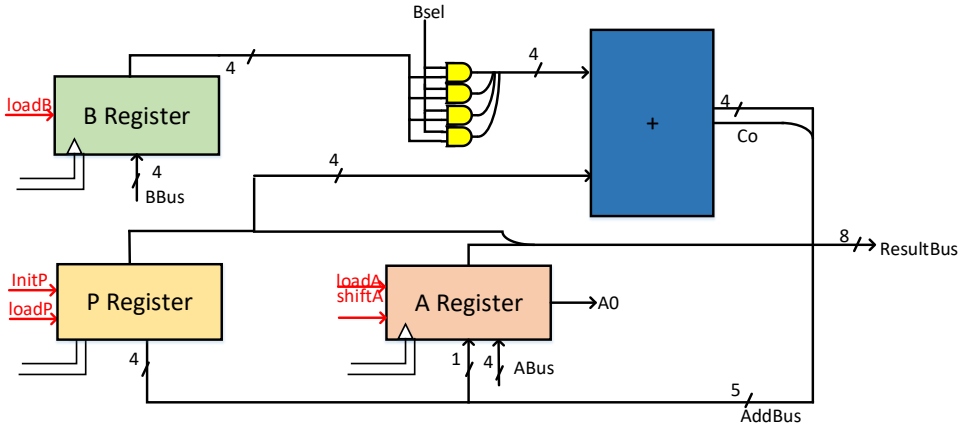






Sequential Multiplier

Shift-Add Multiplier (Datapath SystemVerilog)

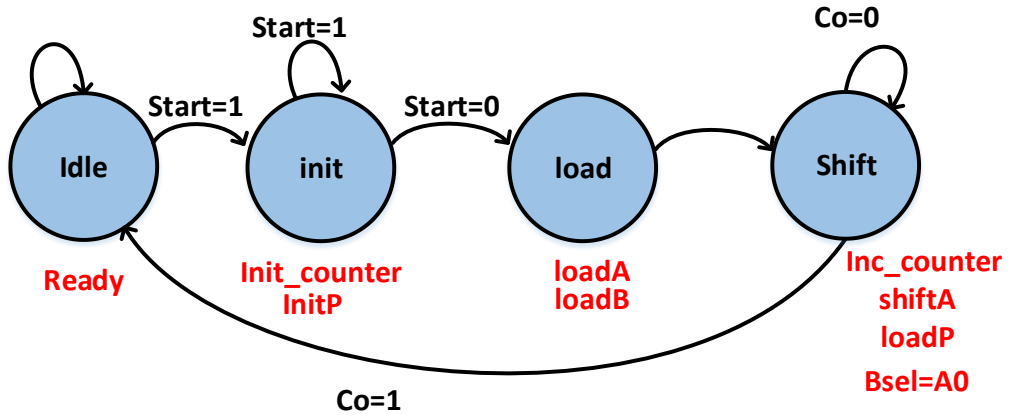


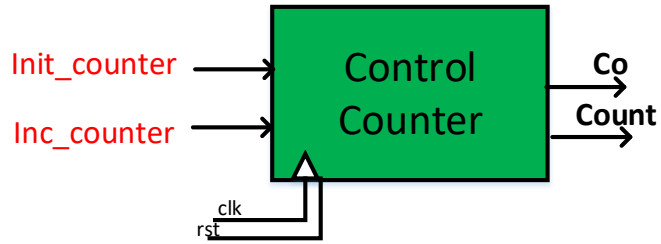

```
module multiplier_DP (
    input  logic        clk, rst,
    input  logic        loadA, loadB, loadP, shiftA, InitP, Bsel,
    input  logic [3:0]  ABus, BBus,
    output logic [7:0]  ResultBus,
    output logic        A0
);

    logic [3:0] Areg, Breg, Preg;
    logic [3:0] B_AND;
    logic [4:0] AddBus;

    always_ff @(posedge clk or posedge rst) begin
        if (rst)
            Breg <= 4'b0;
        else if (loadB)
            Breg <= BBus;
    end

    always_ff @(posedge clk or posedge rst) begin
        if (rst)
            Preg <= 4'b0;
        else if (InitP)
            Preg <= 4'b0;
        else if (loadP)
            Preg <= AddBus[4:1];
    end

    always_ff @(posedge clk or posedge rst) begin
        if (rst)
            Areg <= 4'b0;
        else if (loadA)
            Areg <= ABus;
        else if (shiftA)
            Areg <= {AddBus[0], Areg[3:1]};
    end

    assign B_AND     = Bsel ? Breg : 4'b0;
    assign AddBus    = B_AND + Preg;
    assign ResultBus = {Preg, Areg};
    assign A0        = Areg[0];

endmodule
```

Sequential Multiplier

Shift-Add Multiplier (Controller SystemVerilog)

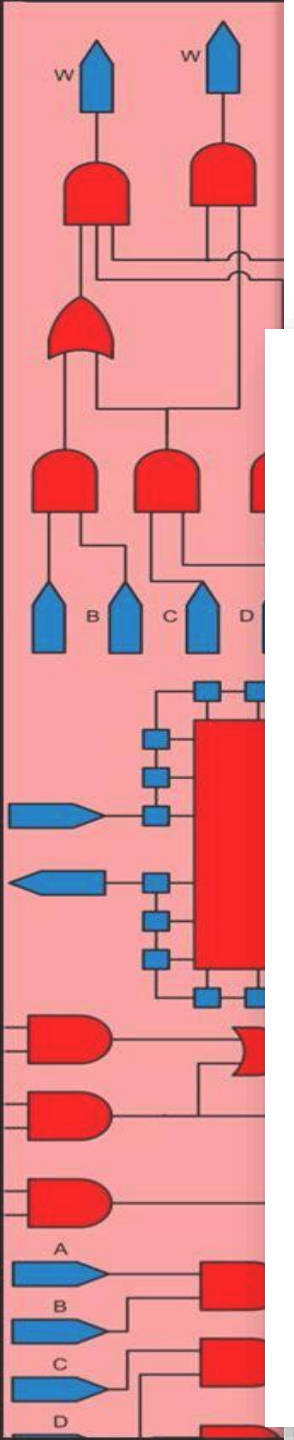
```
module multiplier_CU (
    input  logic        clk,
    input  logic        rst,
    input  logic        start,
    input  logic        A0,
    output logic        loadA, shiftA, loadB, loadP, InitP, Bsel, ready
);

    typedef enum logic [1:0] {
        Idle  = 2'b00,
        Init  = 2'b01,
        load  = 2'b10,
        shift = 2'b11
    } state_t;

    logic Co;
    logic Init_counter, Inc_counter;
    logic [1:0] Count;
    state_t pstate, nstate;
```

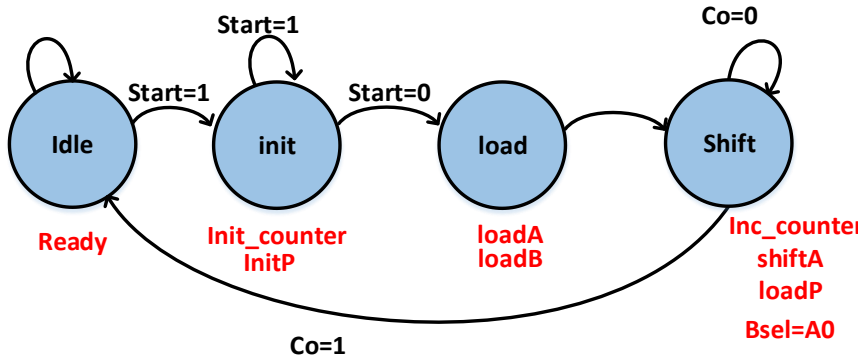


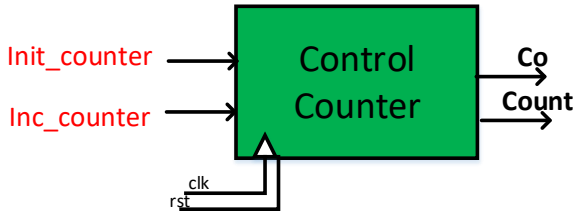


Sequential Multiplier

Shift-Add Multiplier (Controller SystemVerilog)





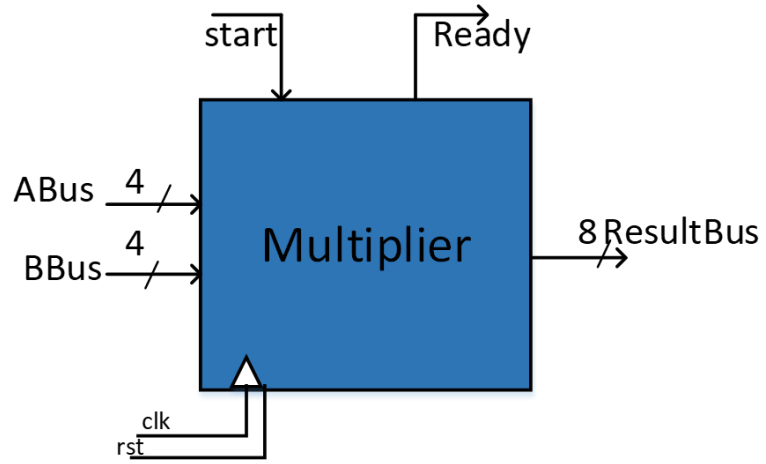



```
always_comb begin
    nstate = Idle;
    {loadA, shiftA, loadB, loadP, InitP, Bsel, ready} = 7'b0;
    {Init_counter, Inc_counter} = 2'b00;

    case (pstate)
        Idle: begin
            nstate = start ? Init : Idle;
            ready  = 1'b1;
        end
        Init: begin
            nstate       = start ? Init : load;
            Init_counter = 1'b1;
            InitP        = 1'b1;
        end
        load: begin
            nstate = shift;
            loadA  = 1'b1;
            loadB  = 1'b1;
        end
        shift: begin
            nstate      = Co ? Idle : shift;
            loadP       = 1'b1;
            shiftA      = 1'b1;
            Inc_counter = 1'b1;
            Bsel        = A0;
        end
        default: begin
            nstate = Idle;
        end
    endcase
end
```

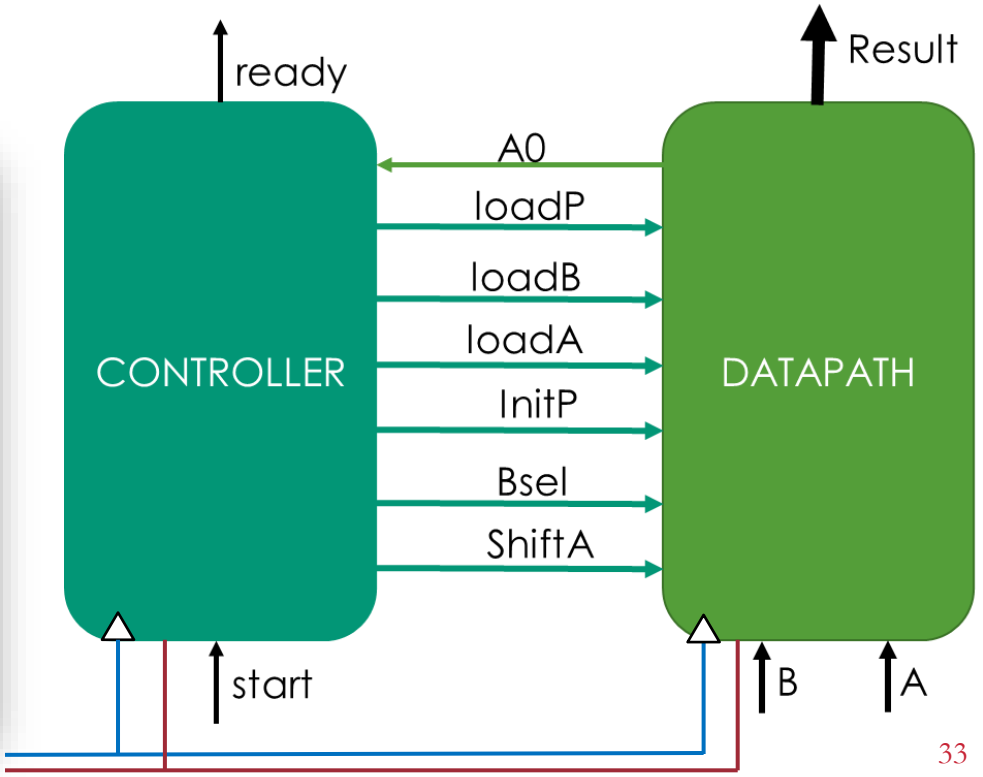
```
    always_ff @(posedge clk or posedge rst) begin
        if (rst)
            pstate <= Idle;
        else
            pstate <= nstate;
    end

    always_ff @(posedge clk or posedge rst) begin
        if (rst)
            Count <= 2'b00;
        else begin
            if (Init_counter)
                Count <= 2'b00;
            else if (Inc_counter)
                Count <= Count + 1;
        end
    end

    assign Co = &Count;

endmodule
```

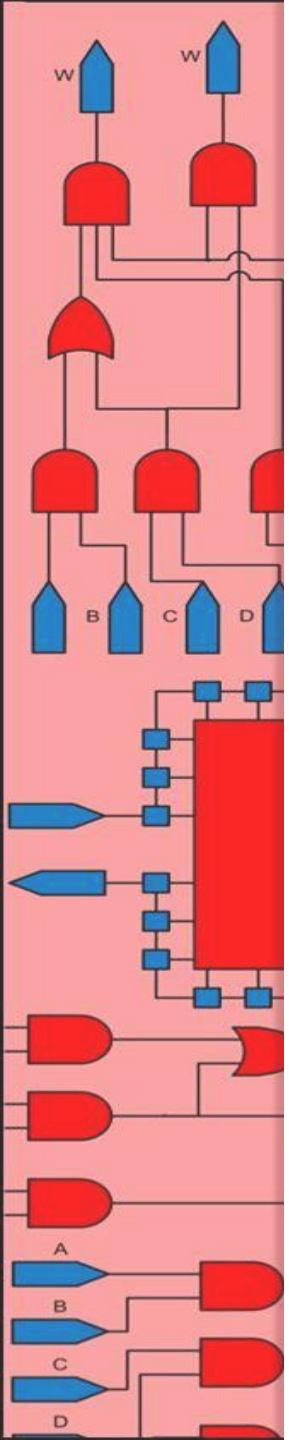
Sequential Multiplier

Shift-Add Multiplier (Top-level SystemVerilog)




```
module multiplier_top (
    input  logic         clk, rst,
    input  logic         start,
    input  logic [3:0]   ABus, BBus,
    output logic [7:0]   ResultBus,
    output logic         ready
);

    logic A0;
    logic loadA, shiftA, loadB, loadP, InitP, Bsel;

    multiplier_DP dp (
        .clk(clk),
        .rst(rst),
        .loadA(loadA),
        .loadB(loadB),
        .loadP(loadP),
        .shiftA(shiftA),
        .InitP(InitP),
        .Bsel(Bsel),
        .ABus(ABus),
        .BBus(BBus),
        .ResultBus(ResultBus),
        .A0(A0)
    );

    multiplier_CU cu (
        .clk(clk),
        .rst(rst),
        .start(start),
        .A0(A0),
        .loadA(loadA),
        .shiftA(shiftA),
        .loadB(loadB),
        .loadP(loadP),
        .InitP(InitP),
        .Bsel(Bsel),
        .ready(ready)
    );

endmodule
```

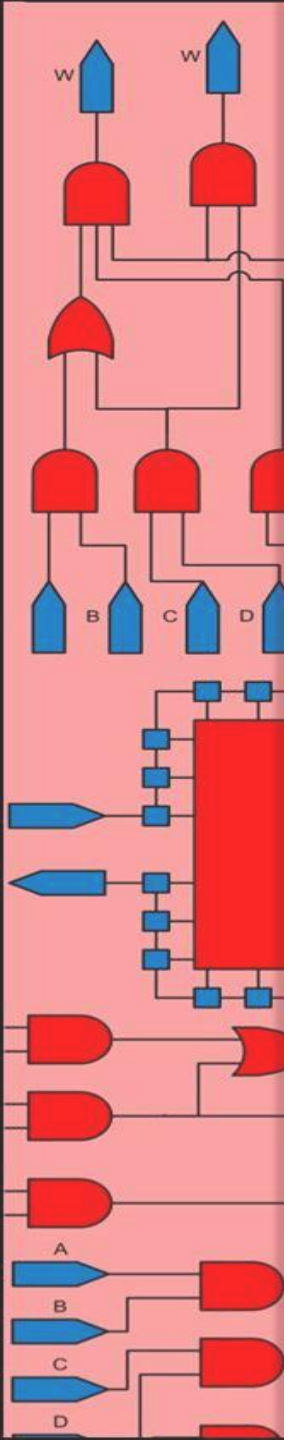




Outline

5- RTL Methodology

- Basic Design Flow
- Serial Adder
- Divider Circuit
- Sequential Multiplier
- **Exponential Accelerator**



Exponential Circuit

$$e^x = 1 + \frac{x^1}{1!} + \frac{x^2}{2!} + \frac{x^3}{3!} + \frac{x^4}{4!} + \frac{x^5}{5!} + \frac{x^6}{6!} + \frac{x^7}{7!} + \ldots$$

Input X is 16 bit-fraction $0 \le X < 1$

$$e^x = \sum_{k=0}^{\infty} \frac{x^k}{k!}$$

```
r = 1; // 1 for result
t = 1; // 1 for term
for( k = 1; k < n; k++ )
{
    t = t × x × ( 1 / k );
    r = r + t;
}
```

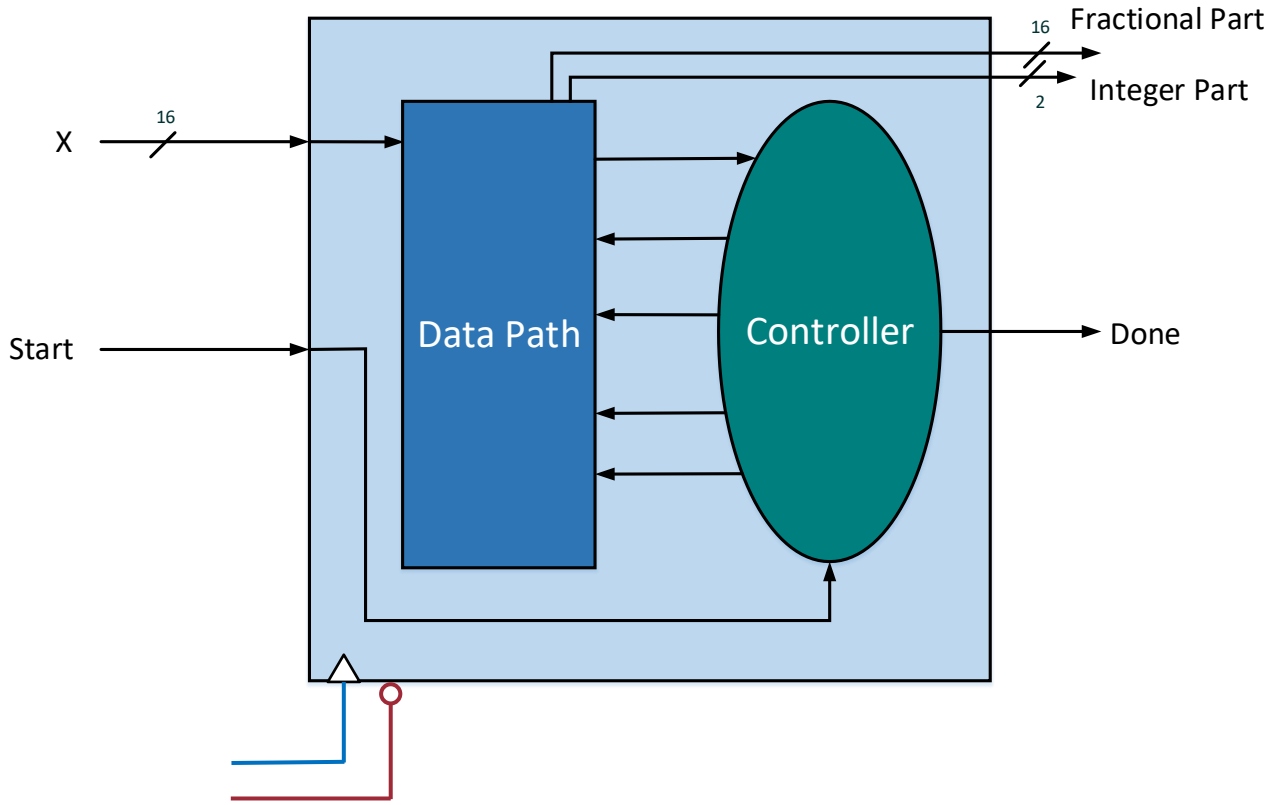
```
r = 1; // 1 for result
t = 1; // 1 for term
for( k = 1; k < n; k++ )
{
    t = t × x
    t = t × ( 1 / k );
    r = r + t;
}
```

Exponential Circuit

$$e^x = 1 + \frac{x^1}{1!} + \frac{x^2}{2!} + \frac{x^3}{3!} + \frac{x^4}{4!} + \frac{x^5}{5!} + \frac{x^6}{6!} + \frac{x^7}{7!} + \ldots$$

Input X is 16 bit-fraction $0 \le X < 1$

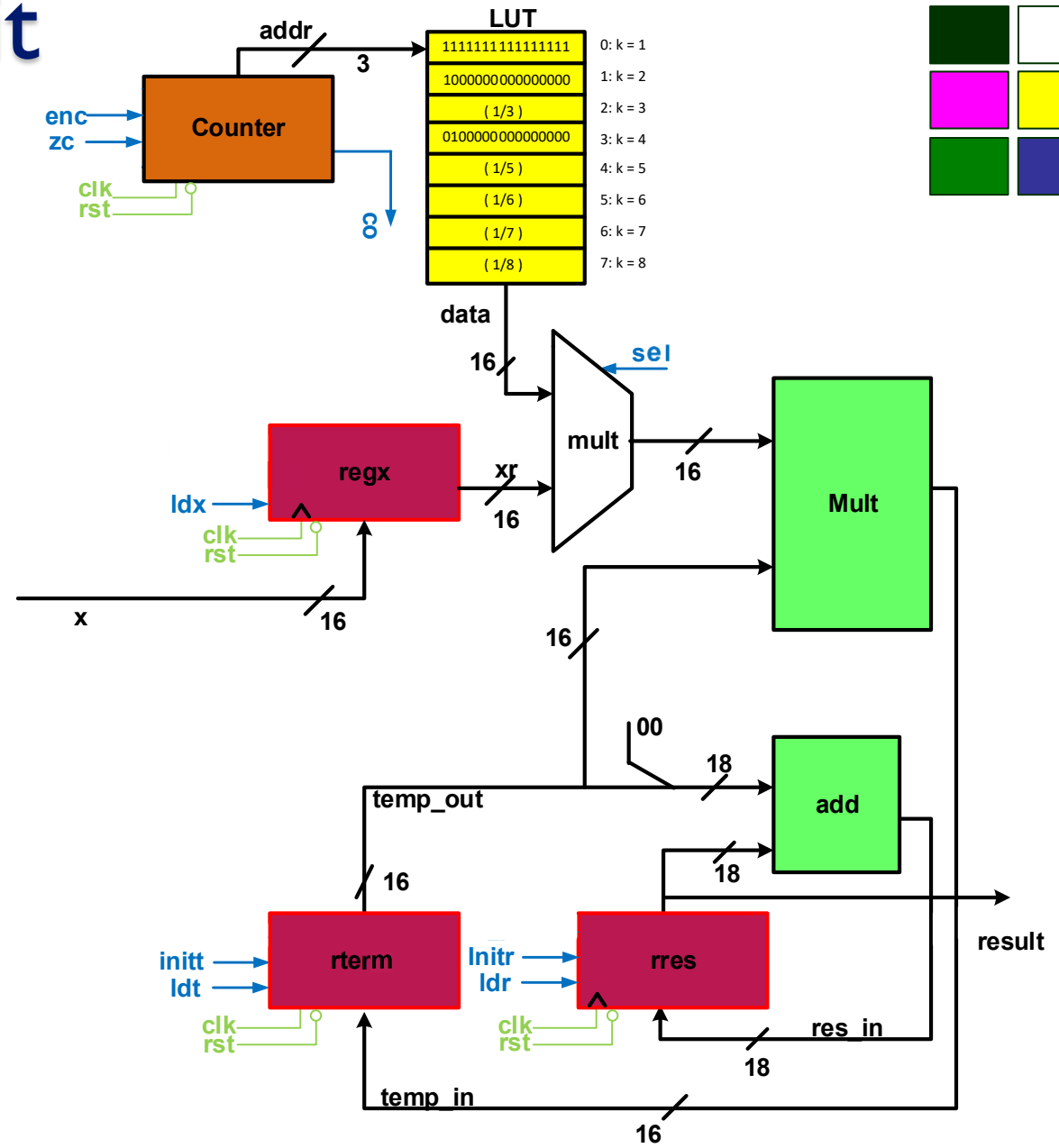
```
r = 1; // 1 for result
t = 1; // 1 for term
for( k = 1; k < n; k++ )
{
    t = t × x
    t = t × ( 1 / k );
    r = r + t;
}
```



Exponential Circuit

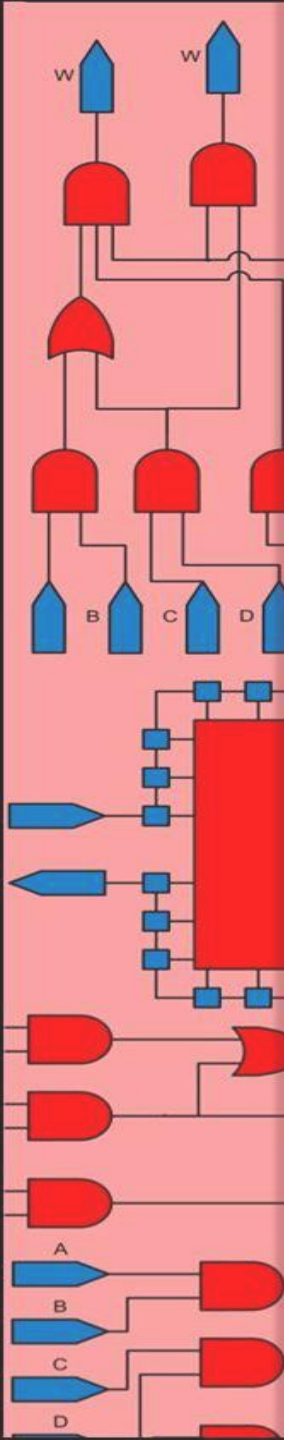
Datapath

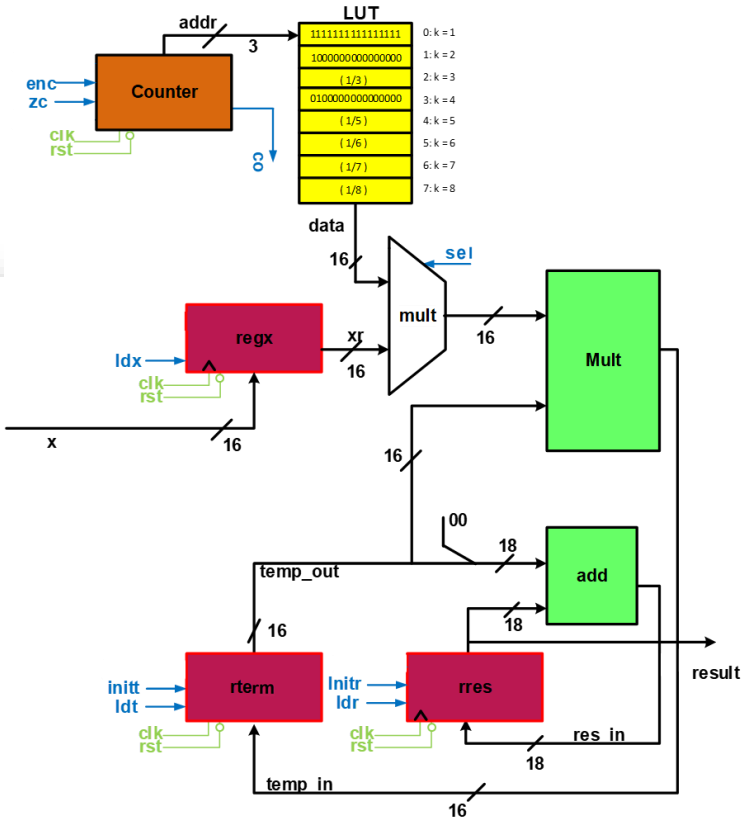
```
r = 1; // 1 for result
t = 1; // 1 for term
for( k = 1; k < n; k++ )
{
    t = t × x
    t = t × ( 1 / k );
    r = r + t;
}
```



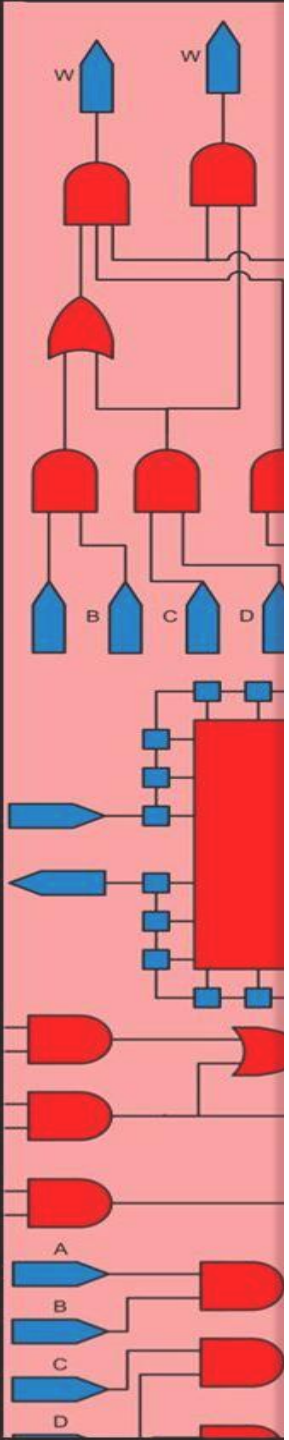
Exponential Circuit

Datapath (SystemVerilog)



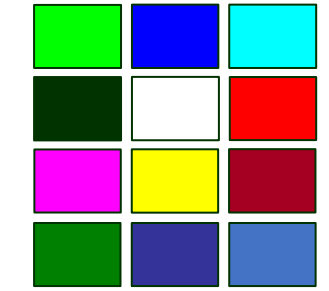



```
module exp_DP (
    input  logic         clk,
    input  logic         rst_n,
    input  logic         ldx, sel,
    input  logic         initt, ldt,
    input  logic         initr, ldr,
    input  logic         zc, enc,
    input  logic [15:0]  x,
    output logic         co,
    output logic [17:0]  result
);
    logic [15:0] data, m_out, term_in;
    logic [15:0] term_out, xr;
    logic [17:0] res_in;
    logic [2:0]  addr;

    register   #(.BIT_WIDTH(16)) regx    (.clk(clk), .rst_n(rst_n), .init0(), .init1(), .load(ldx),
                                          .in_value(x), .out_value(xr));
    counter    #(.BIT_WIDTH(3))  counter (.clk(clk), .rst_n(rst_n), .init0(zc), .enable(enc),
                                          .count(addr), .co(co));
    LUT                          lut     (.addr(addr), .data(data));
    mux2to1    #(.BIT_WIDTH(16)) mux     (.a(xr), .b(data), .sel(sel), .out_value(m_out));
    multiplier #(.BIT_WIDTH(16)) mult    (.a(m_out), .b(term_out), .out_value(term_in));
    register   #(.BIT_WIDTH(16),
          .INIT_VALUE(16'hFFFF)) rterm   (.clk(clk), .rst_n(rst_n), .init0(), .init1(initt), .load(ldt),
                                          .in_value(term_in), .out_value(term_out));
    adder      #(.BIT_WIDTH(18)) add     (.a({2'b00, term_out}), .b(result), .sum(res_in), .c_out());
    register   #(.BIT_WIDTH(18),
         .INIT_VALUE(18'h10000)) rres    (.clk(clk), .rst_n(rst_n), .init0(), .init1(initr), .load(ldr),
                                          .in_value(res_in), .out_value(result));
endmodule
```



Exponential Circuit

Datapath (SystemVerilog)

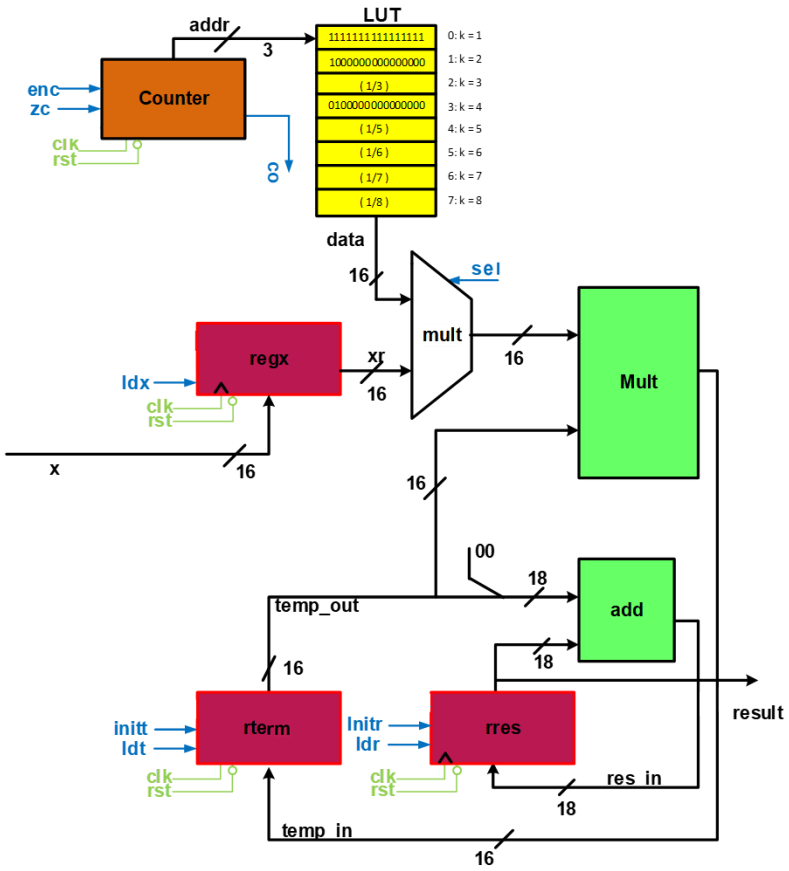


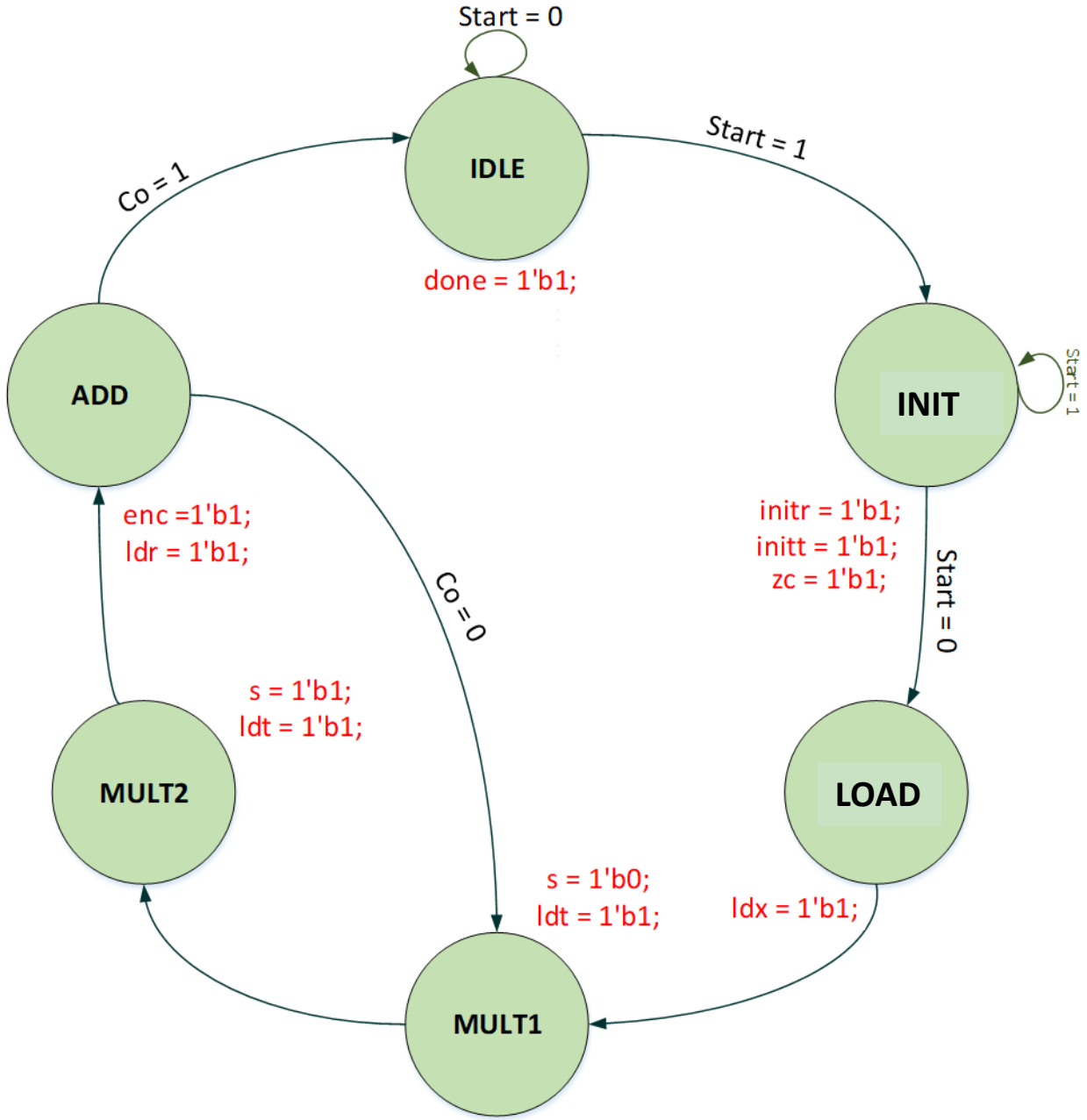
LUT

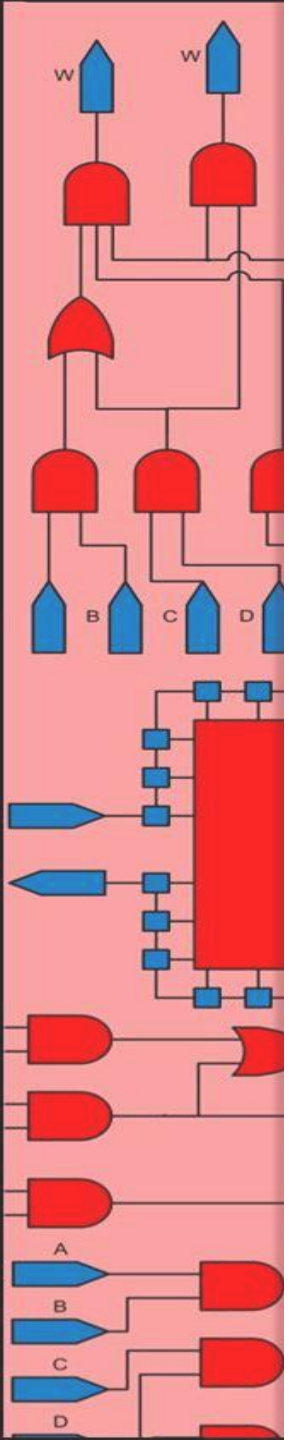
```
module LUT (
    input  logic [2:0]  addr,
    output logic [15:0] data);

    always_comb begin
        data = 16'h0000;

        case(addr)
            0: data = 16'hFFFF;
            1: data = 16'h8000; // 1/2
            2: data = 16'h5555; // 1/3
            3: data = 16'h4000; // 1/4
            4: data = 16'h3333; // 1/5
            5: data = 16'h2AAA; // 1/6
            6: data = 16'h2492; // 1/7
            7: data = 16'h2000; // 1/8
        endcase
    end

endmodule
```

Exponential Circuit

Controller



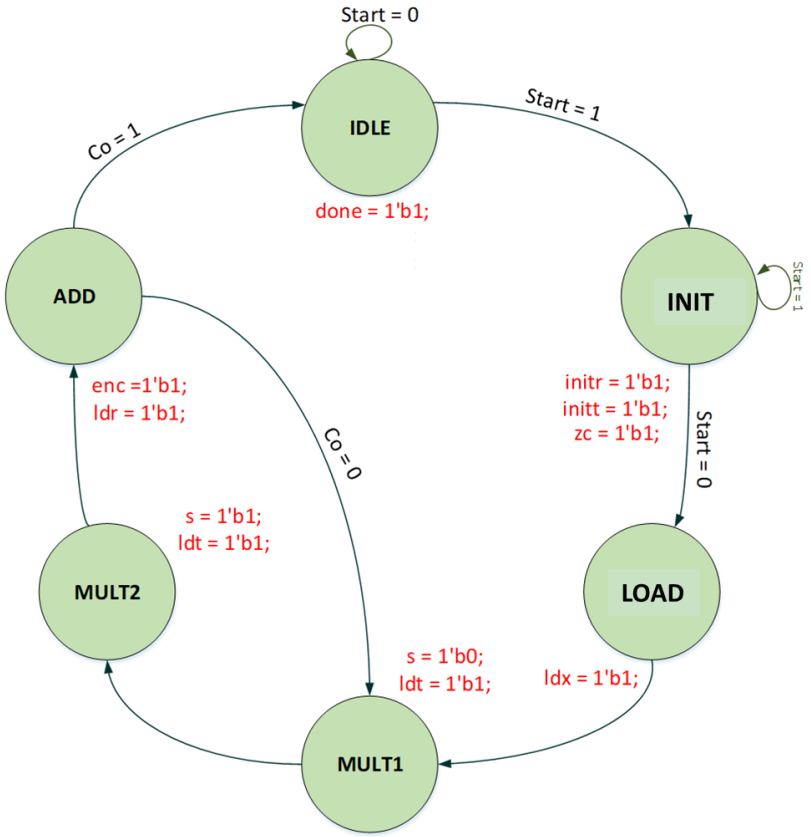




Exponential Circuit

Controller (SystemVerilog)

```
module exp_CU (
    input  logic clk,
    input  logic rst_n,
    input  logic start,
    input  logic co,
    output logic done,
    output logic ldx,
    output logic initt, ldt,
    output logic initr, ldr,
    output logic zc, enc, sel
);

    typedef enum logic [2:0] {
        IDLE          = 3'd0,
        INIT          = 3'd1,
        LOAD          = 3'd2,
        MULT1         = 3'd3,
        MULT2         = 3'd4,
        ADD           = 3'd5
    } state_t;

    state_t present_state, next_state;
```

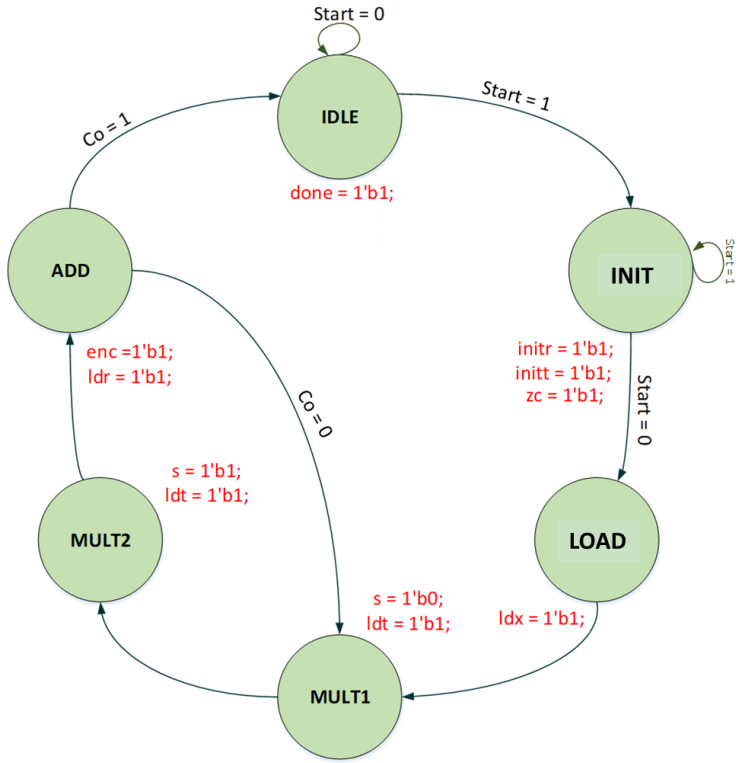


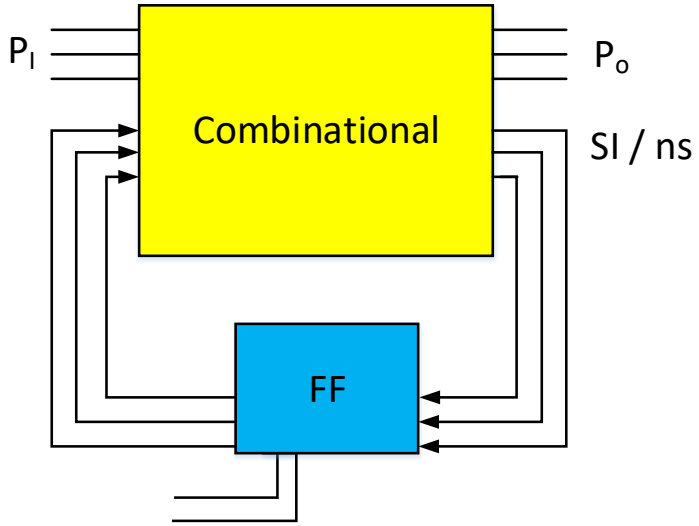
Exponential Circuit

Controller (SystemVerilog)

```
always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n)
        present_state <= IDLE;
    else
        present_state <= next_state;
end
```

```
always_comb begin
    next_state = IDLE;

    case (present_state)
        IDLE:  next_state = (!start) ? IDLE : INIT;
        INIT:  next_state = (start)  ? INIT : LOAD;
        LOAD:  next_state = MULT1;
        MULT1: next_state = MULT2;
        MULT2: next_state = ADD;
        ADD:   next_state = (co) ? IDLE : MULT1;
        default: next_state = IDLE;
    endcase
end
```

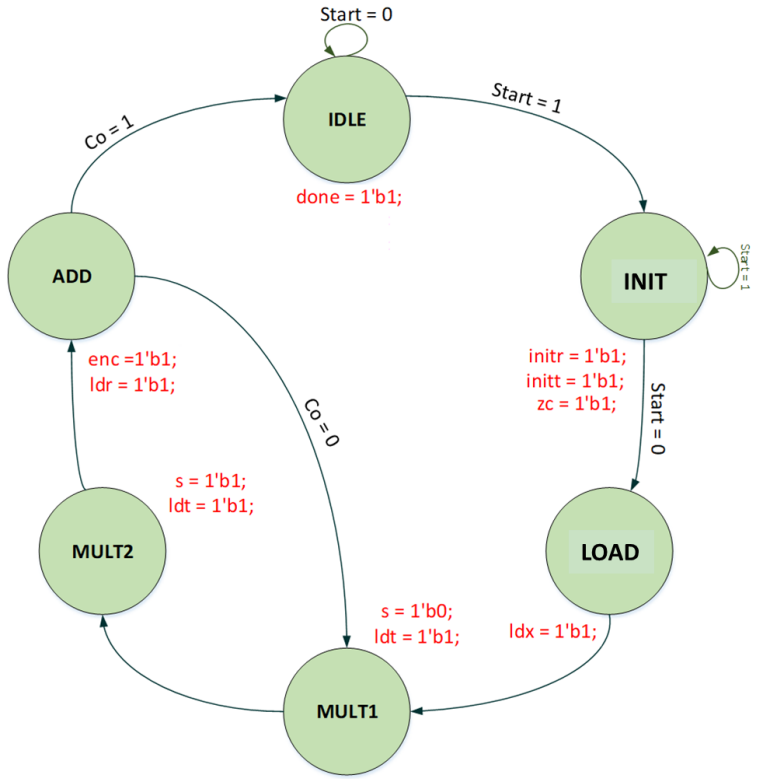


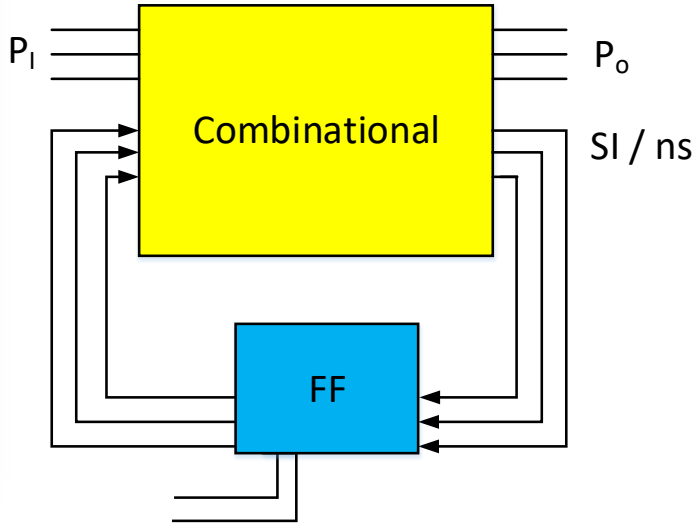


Exponential Circuit

Controller (SystemVerilog)

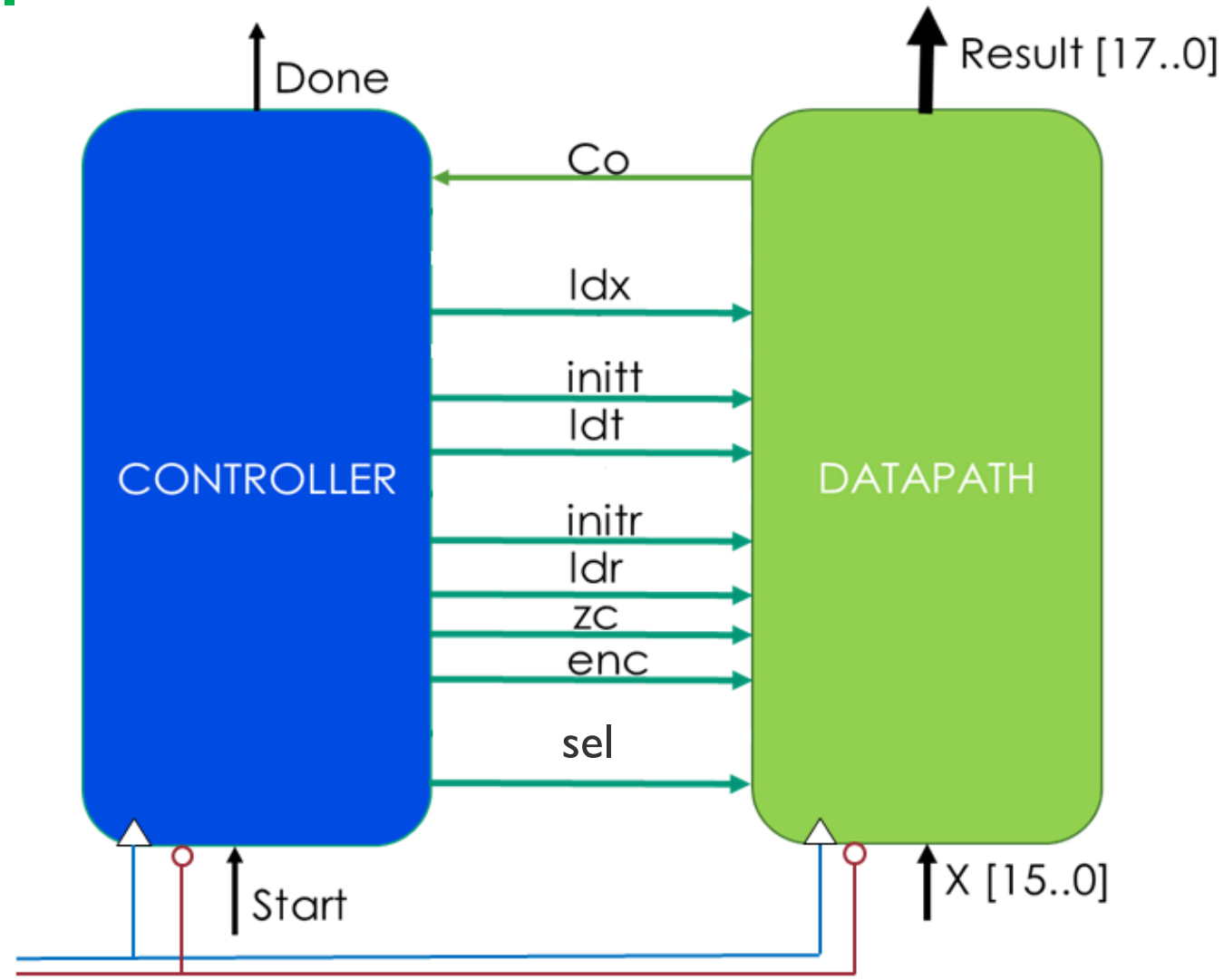
```
always_comb begin
    done   = 1'b0;
    ldx    = 1'b0; sel  = 1'b0;
    initt  = 1'b0; ldt  = 1'b0;
    initr  = 1'b0; ldr  = 1'b0;
    zc     = 1'b0; enc  = 1'b0;

case (present_state)
    IDLE: begin
        done  = 1'b1;
    end

    INIT: begin
        zc    = 1'b1;
        initr = 1'b1;
        initt = 1'b1;
    end

    LOAD: begin
        ldx = 1'b1;
    end

    MULT1: begin
        sel = 1'b0;
        ldt = 1'b1;
    end

    MULT2: begin
        sel = 1'b1;
        ldt = 1'b1;
    end

    ADD: begin
        ldr = 1'b1;
        enc = 1'b1;
    end

    default: begin
        done   = 1'b0;
        ldx    = 1'b0; sel  = 1'b0;
        initt  = 1'b0; ldt  = 1'b0;
        initr  = 1'b0; ldr  = 1'b0;
        zc     = 1'b0; enc  = 1'b0;
    end
endcase
```





Exponential Circuit

Top Level

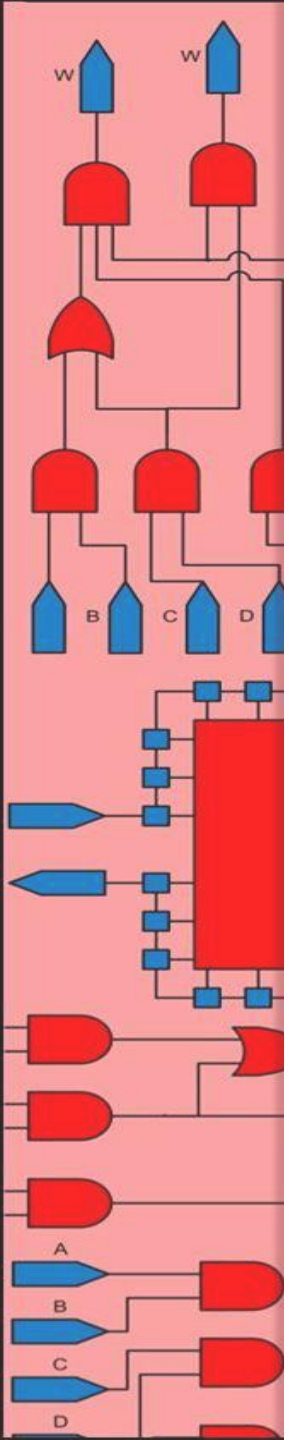


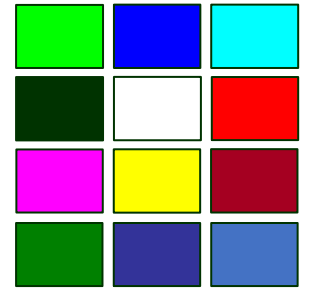
Exponential Circuit

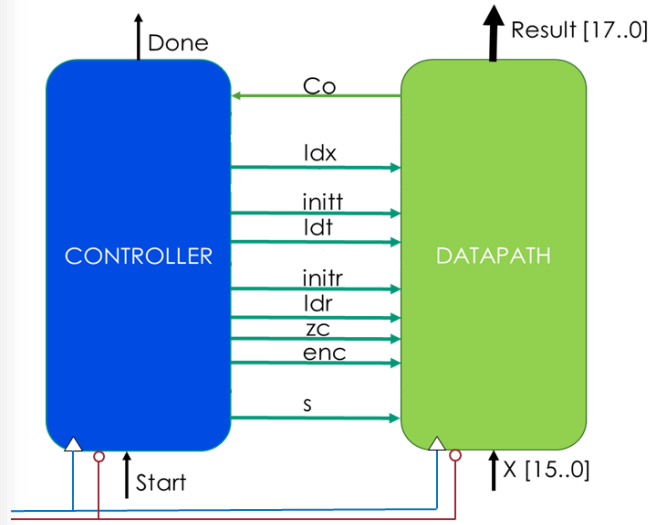
Top Level

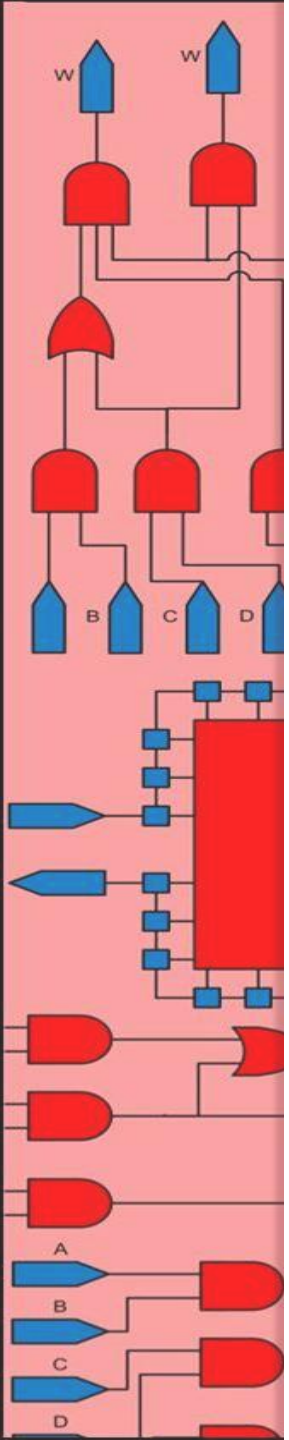
```
module exp_top (
    input  logic clk,
    input  logic rst_n,
    input  logic start,
    input  logic [15:0] x,
    output logic done,
    output logic [1:0] intpart,
    output logic [15:0] fracpart
);

    logic ldx, sel;
    logic initt, ldt;
    logic initr, ldr;
    logic zc, enc, co;
```

```
    exp_DP datapath (
        .clk        (clk),
        .rst_n      (rst_n),
        .ldx        (ldx),
        .initt      (initt),
        .ldt        (ldt),
        .initr      (initr),
        .ldr        (ldr),
        .zc         (zc),
        .enc        (enc),
        .sel        (sel),
        .x          (x),
        .co         (co),
        .result     ({intpart, fracpart})
    );

    exp_CU control (
        .clk        (clk),
        .rst_n      (rst_n),
        .start      (start),
        .co         (co),
        .done       (done),
        .ldx        (ldx),
        .initt      (initt),
        .ldt        (ldt),
        .initr      (initr),
        .ldr        (ldr),
        .zc         (zc),
        .enc        (enc),
        .sel        (sel)
    );
endmodule
```









Conclusion

We have covered these topics:

- Basic RTL design flow
- Serial Adder
- Sequential Divider
- Sequential Multiplier
- Exponential Accelerator